

NOVA

IMS

Information
Management
School

data with **purpose**

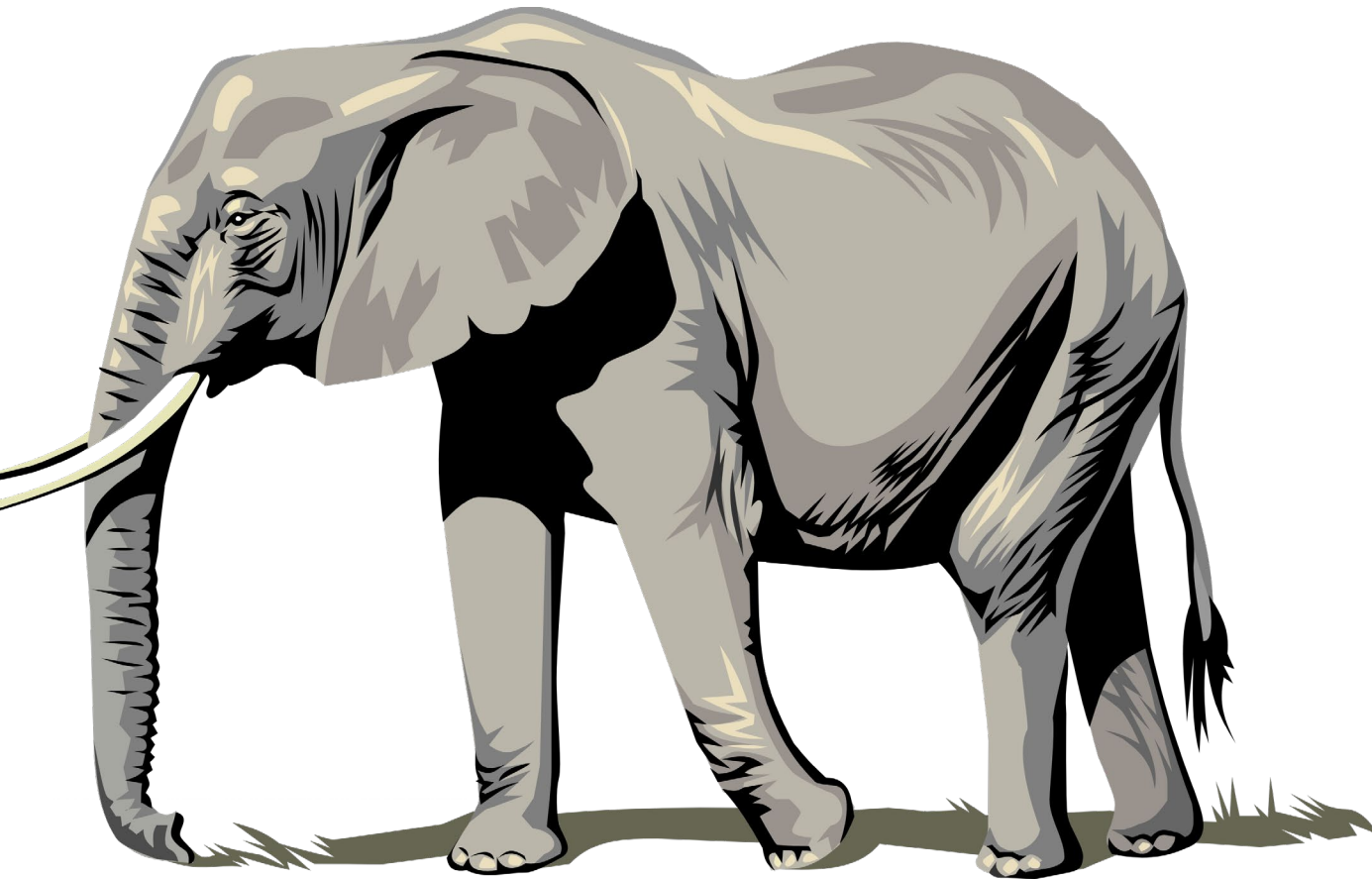
Big Data Analytics

Streaming

Lecturer:

Márcia Lourenço Baptista

m.baptista@novaims.unl.pt



Agenda for Today



- Data Streaming: define and talk about data streaming in large scale
 - What is Data Streaming?
 - Real Time Processing Systems
 - Batch Processing
 - Bound Data (Latency, Throughput, Batching, Disadvantage)
 - Stream Processing
 - What is a Stream?
 - Unbound Data
 - Real World = Unbound Data
 - Applications of Stream Processing
 - Architecture of Stream Processing
 - Real-Time Stream Processor
 - Fault Tolerance, Load Balancing, Unification of Workload, Data Analytics
- Spark Streaming



What is Data Streaming?

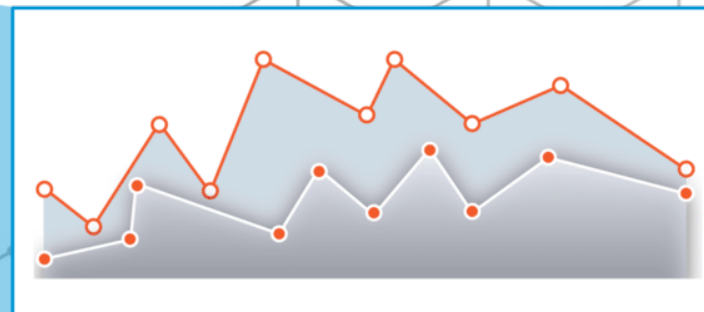
- Technique for transferring **real-time data** as a continuous stream.
- Streaming is important with the growth of the Internet.



User



Streaming Sources



Live Stream Data

*"Without stream processing
there's no big data and no
Internet of Things" – Dana
Sandy, SQLstream*

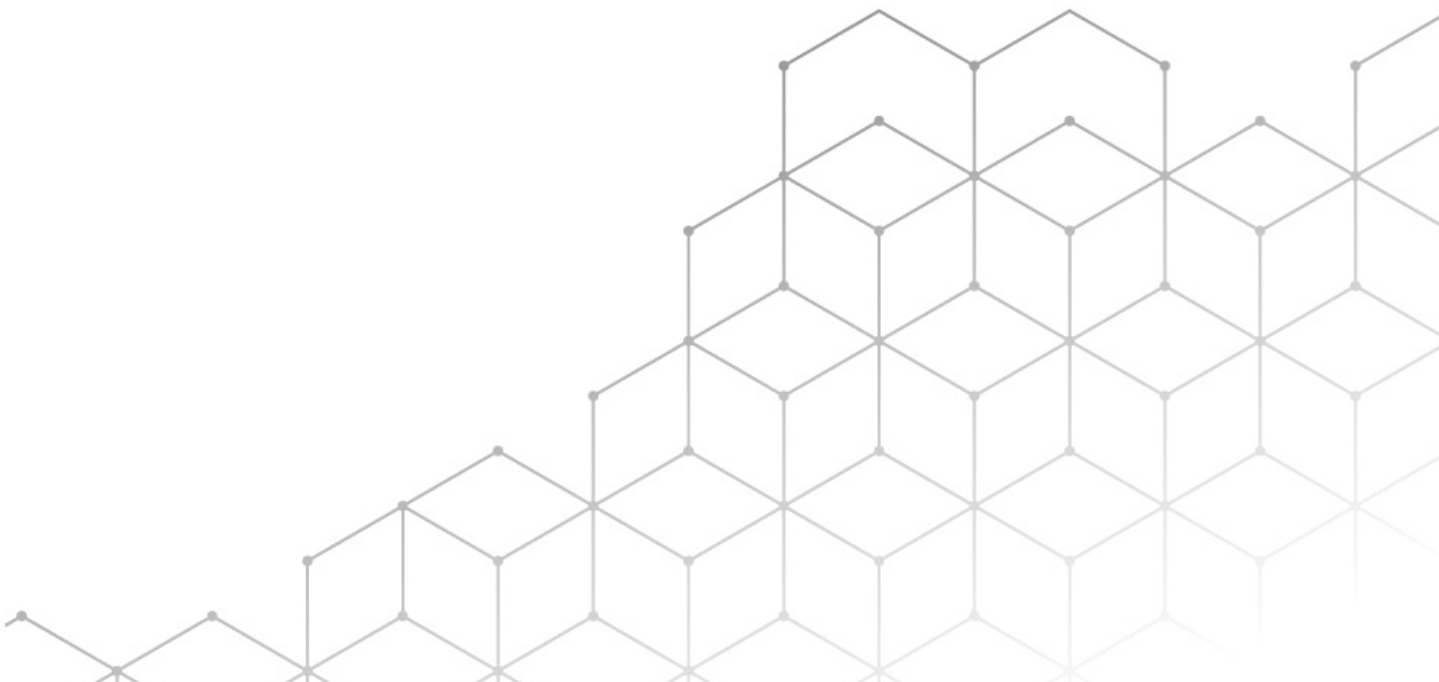
Visual Metaphor

- **Streaming** (in general) just means sending or receiving something in a continuous flow, instead of in one big chunk. It could be *anything* — video, music, even air.

Sender



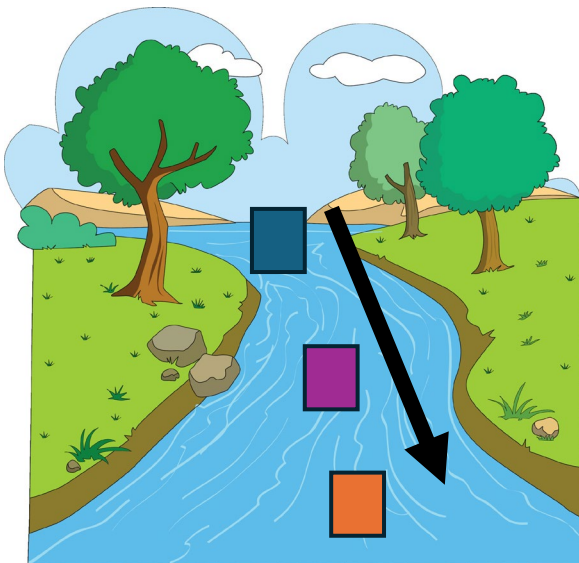
Receiver



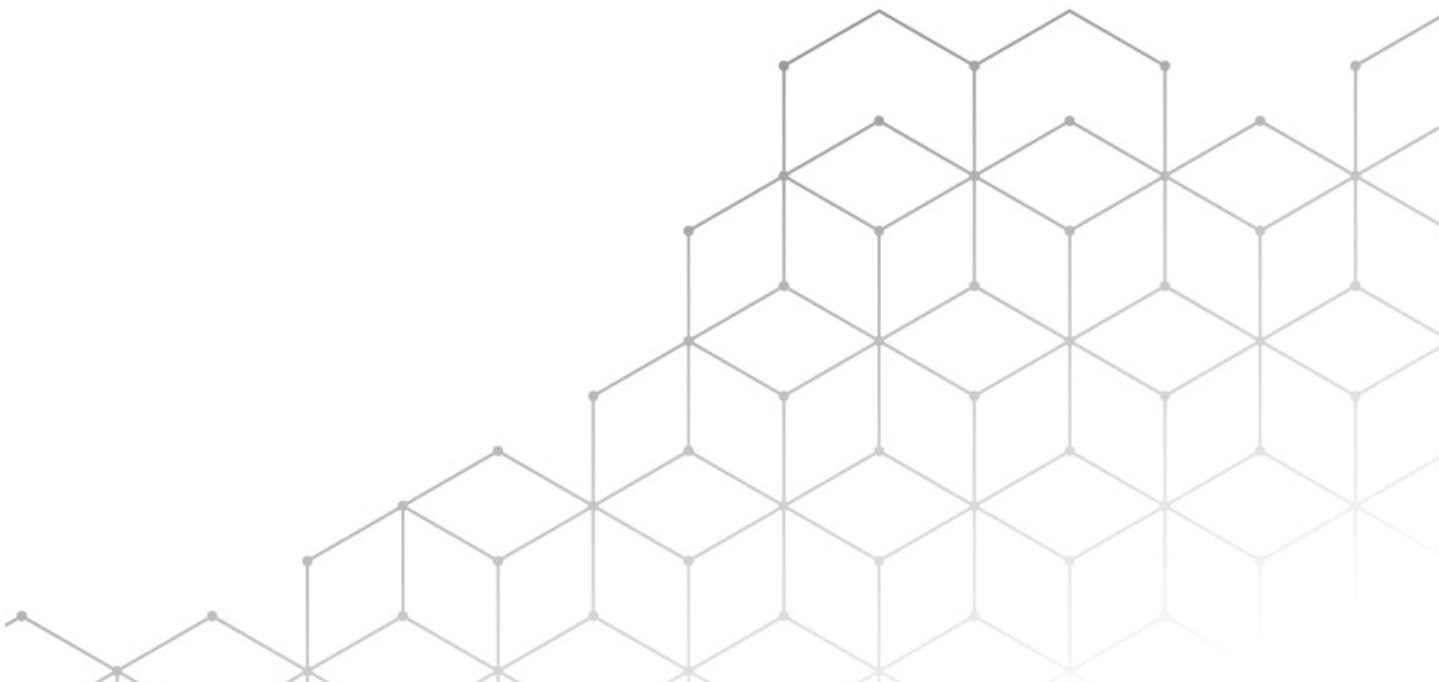
Visual Metaphor

- **Streaming** (in general) just means sending or receiving something in a continuous flow, instead of in one big chunk. It could be *anything* — video, music, even air.

Sender



Receiver



Real Time Processing Systems

- Real-time systems guarantees that the **reaction** to data will be within a tight real-world deadline, usually seconds or milliseconds.

Stock Market



ATM

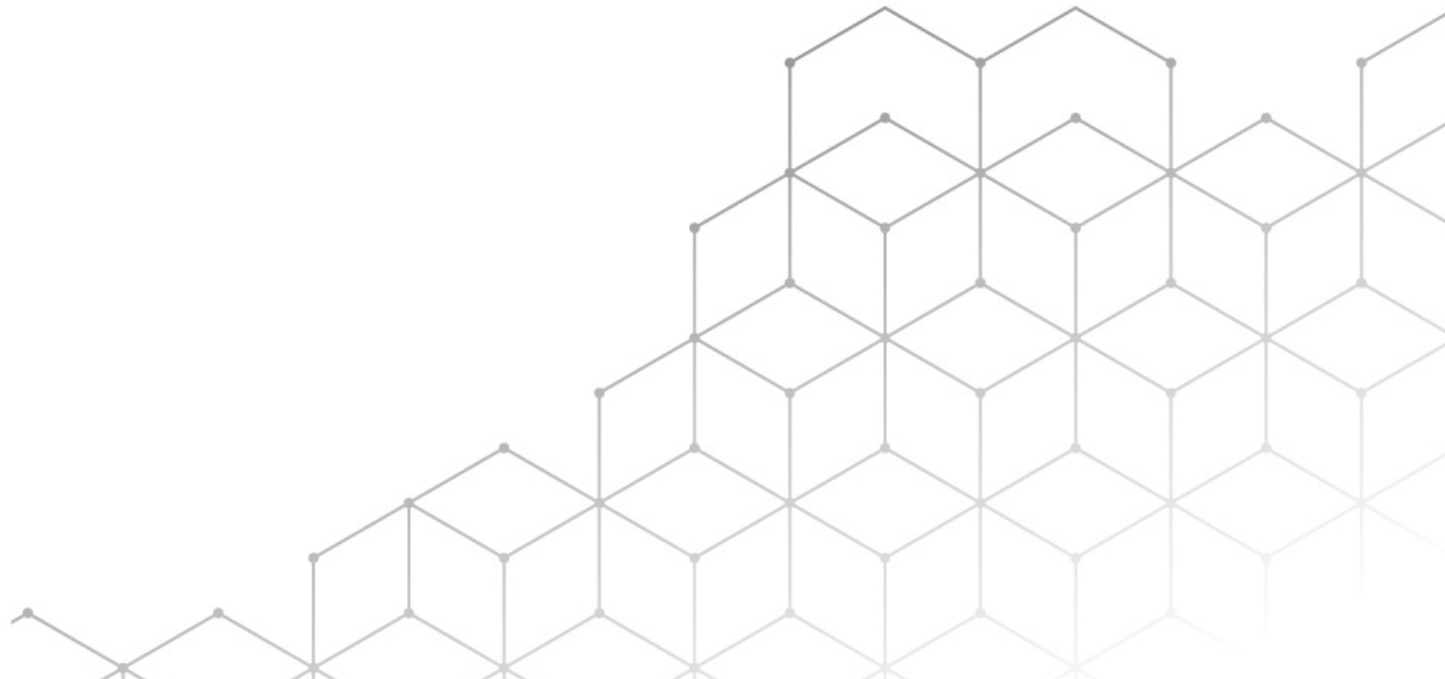


Air Traffic Control



Anti-Lock Bracking Systems

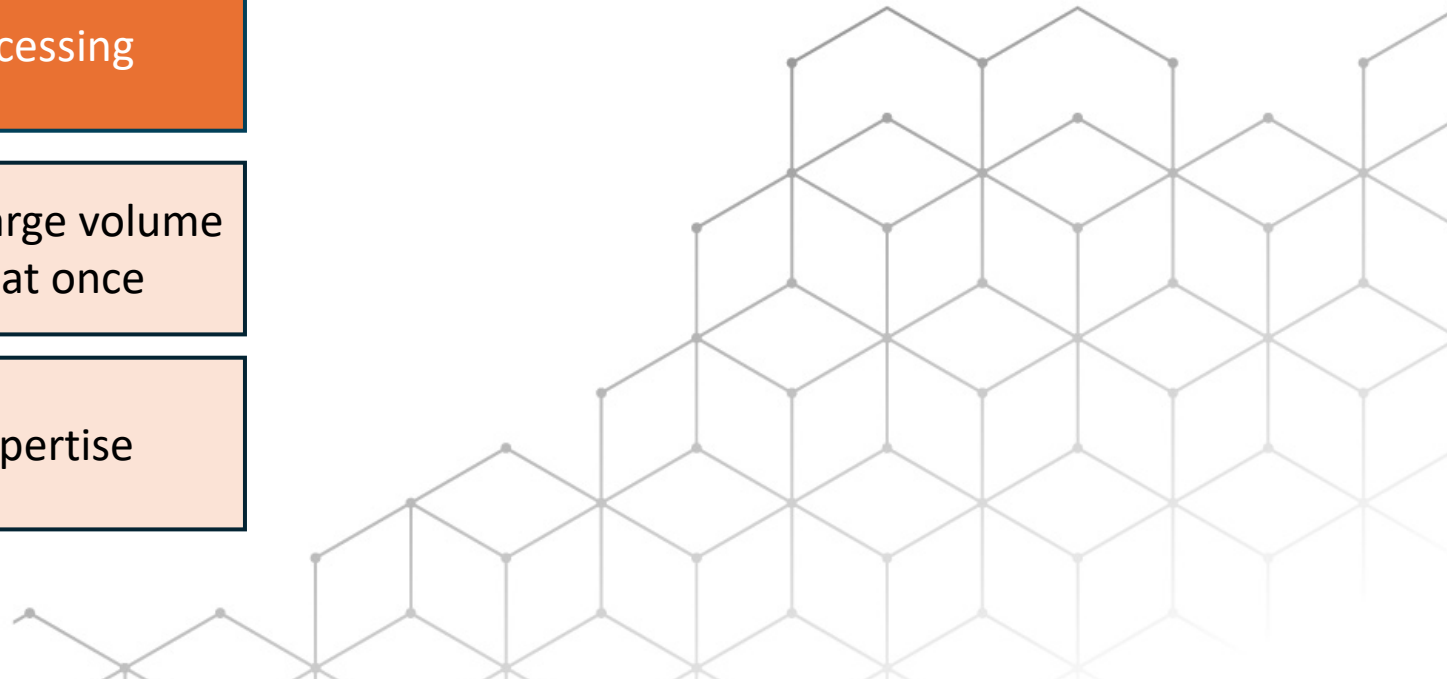
- The vehicle safety advantage of automated and optimized braking requires real-time data. Information about tire-to-road conditions have to be immediate.



Batch Processing

- Batch processing is the processing of large volume of data **all at once**.
- An example of a batch processing job is all the transactions a financial firm might submit at the end of the week.

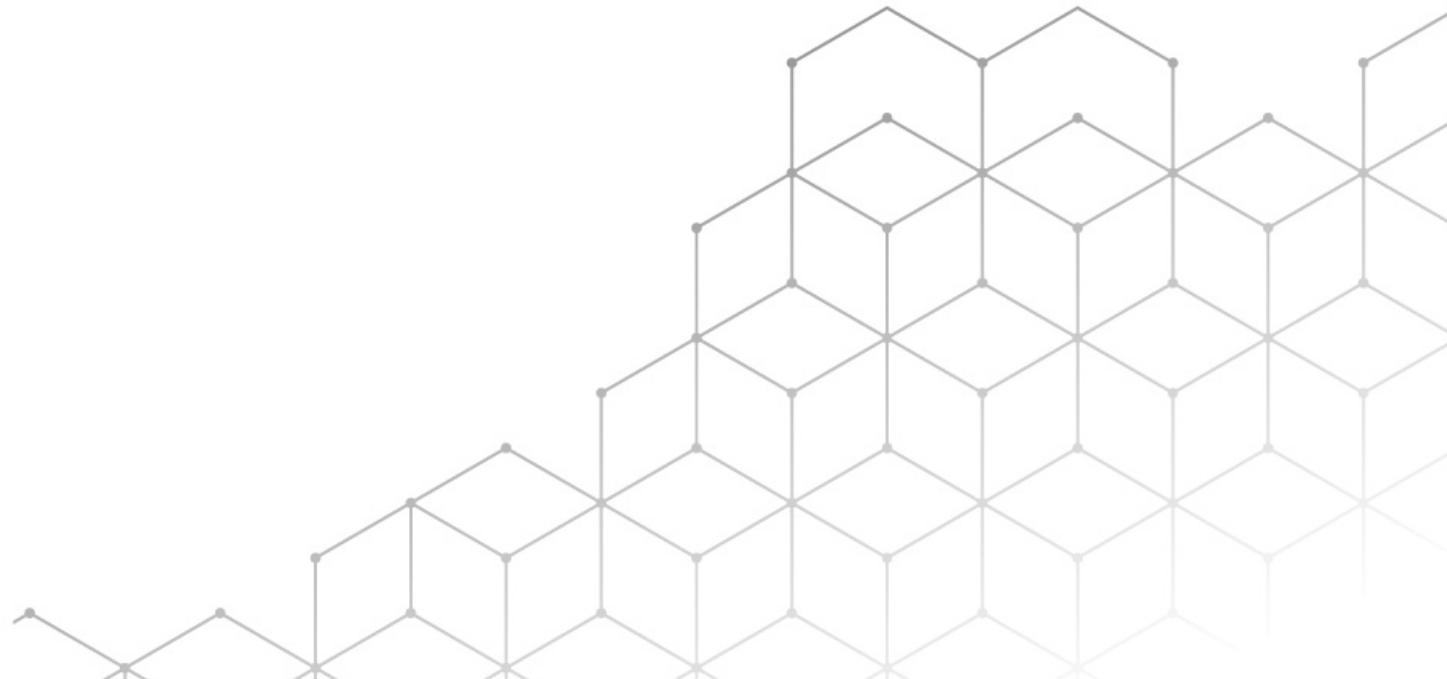
Real-time Processing	Batch Processing
Reacts to data in seconds or milliseconds	Processes to large volume of data all at once
Difficult to Implement	Coding Expertise



Example of Batch Processing

Imagine doing laundry:

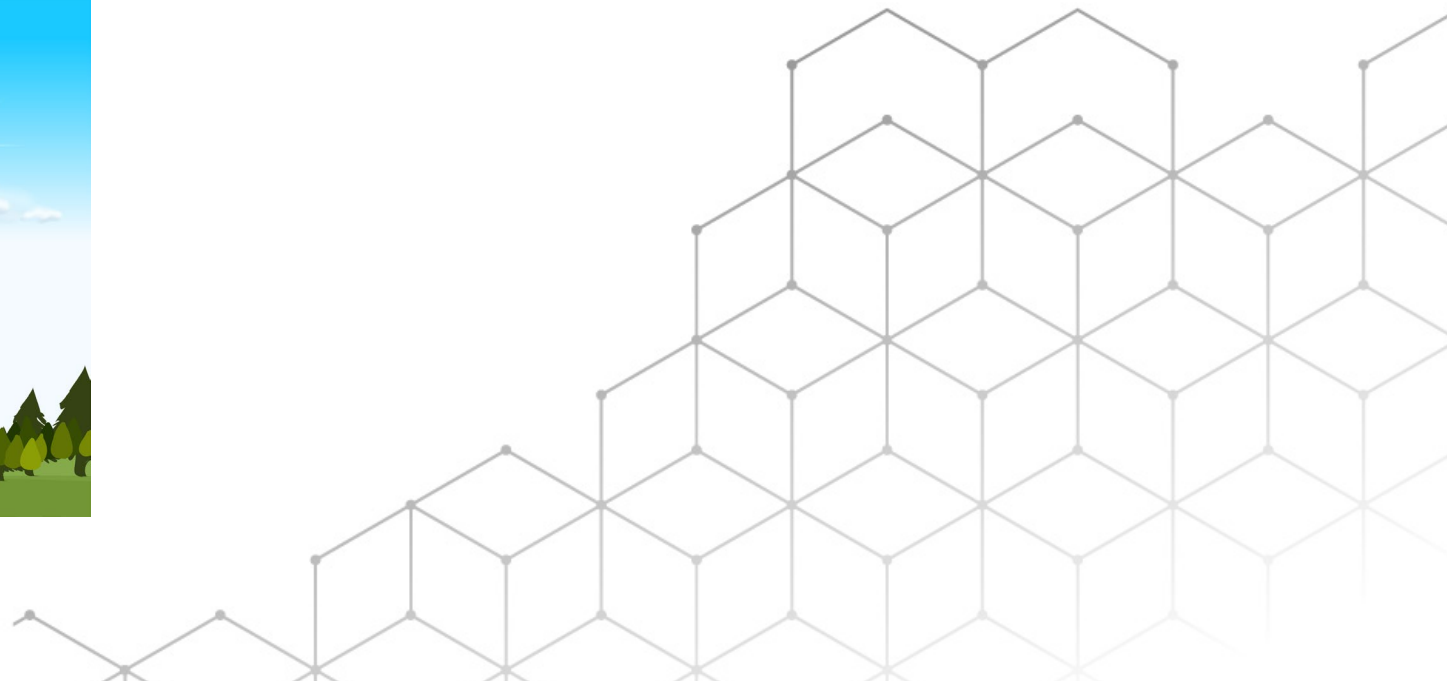
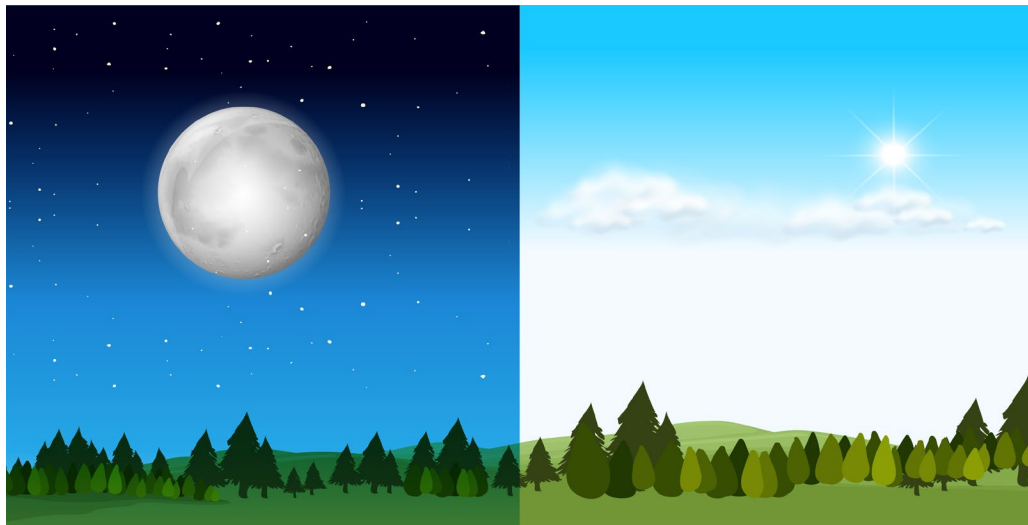
- You wait until you have a full load.
- Then you wash it all together. That's batch processing — efficient, but not real-time.



Other Examples

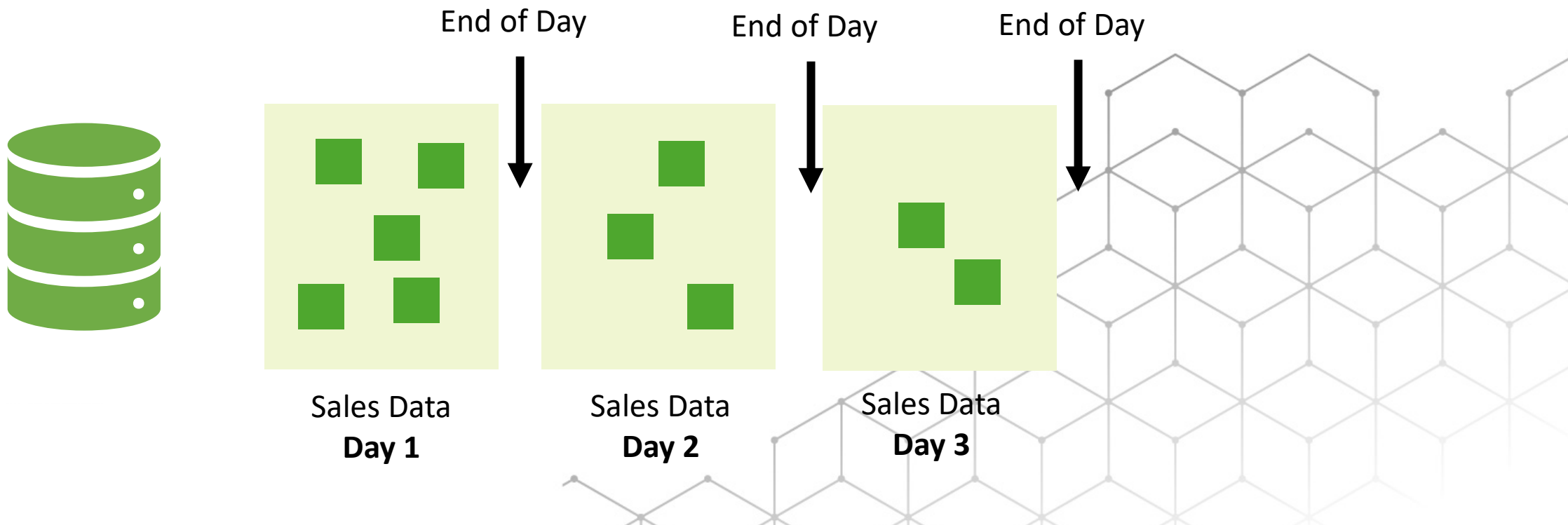
In computing:

- You might process all sales data at the end of the day.
- Or run reports every night when the system is less busy.



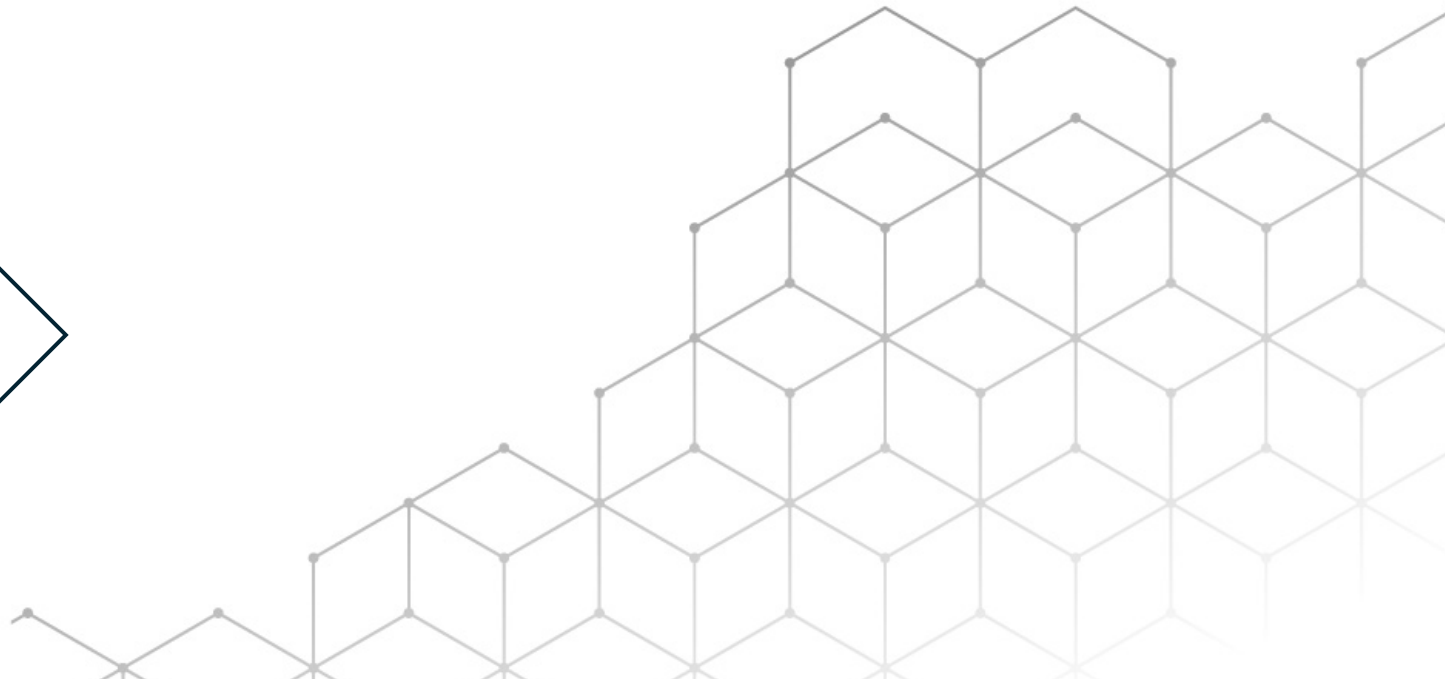
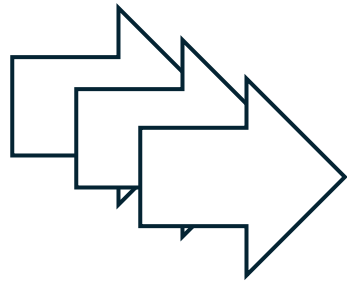
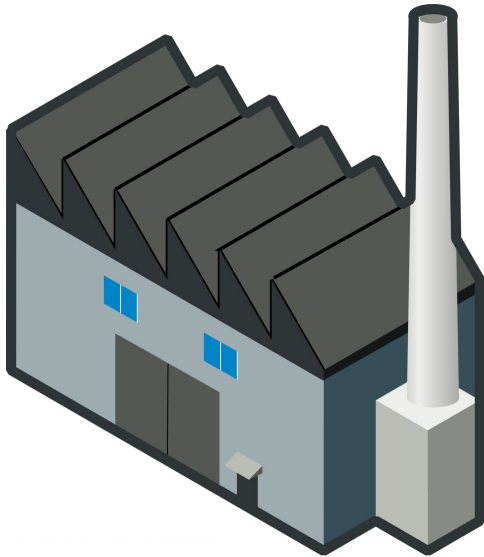
Bound Data

- **Bound data** is finite and unchanging data, where everything is known about the set of data.
- Traditionally we have analysed bound data looking back into the past.



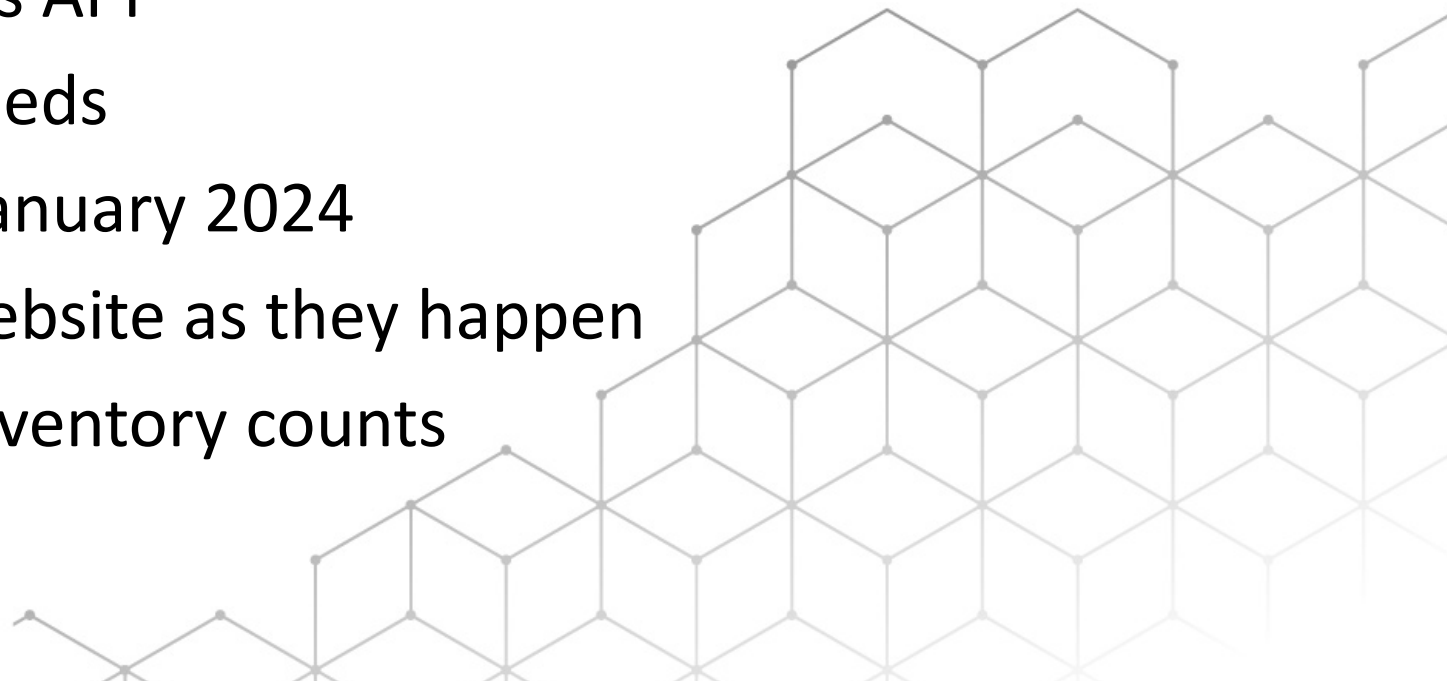
Example of Bound Data

- Processing a static CSV file with 1000 sales records = bound data.
- Processing real-time sensor data from a factory = unbound data (if you do not set limits)



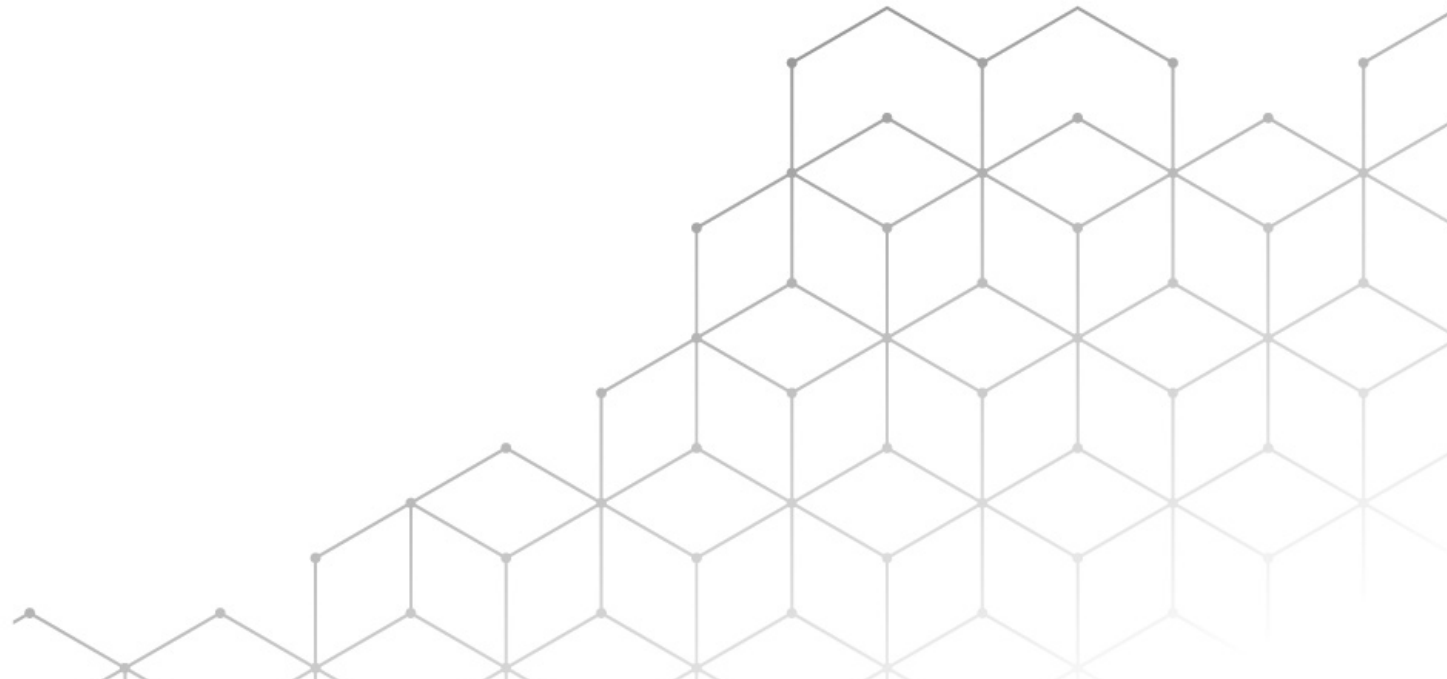
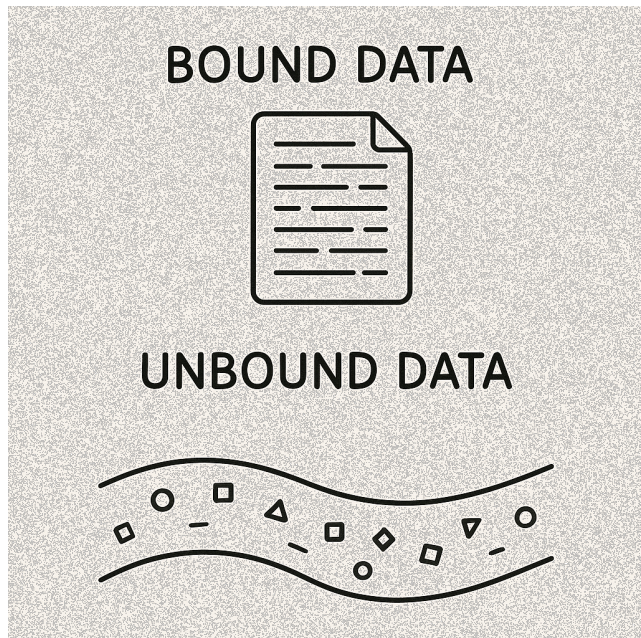
Pick the Examples

- A CSV file with 1,000 customer records
- Temperature data from IoT sensors
- A monthly sales report downloaded from a database
- Tweets coming from Twitter's API
- Live security camera video feeds
- Historical weather data for January 2024
- Clicks or user activity on a website as they happen
- An Excel spreadsheet with inventory counts



Data Bounded and Unbounded

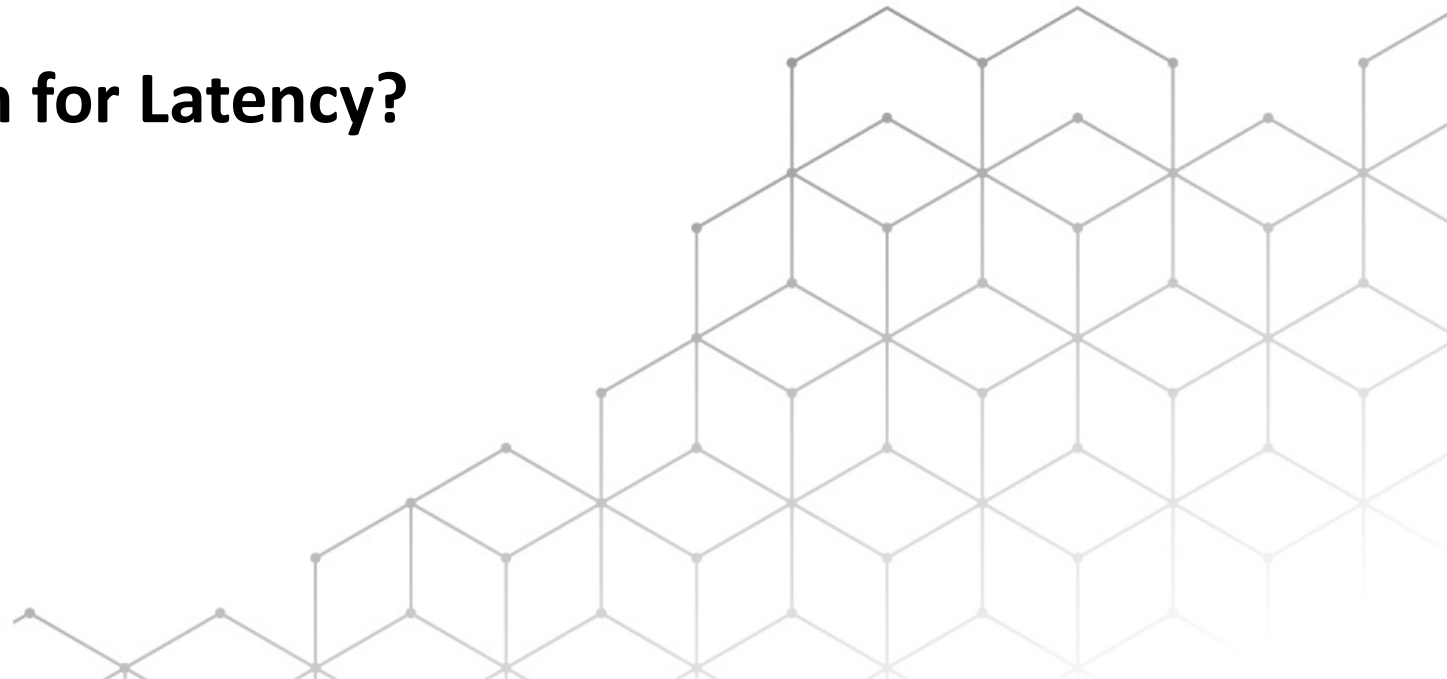
- 🧠 Think of it as data with edges
- 🧠 Think of it as data with no boundaries — it just keeps flowing.



Latency

- Latency is a measure of time delay in a system.
- **Network latency:** time it takes for a message to reach the server.
- **Server Latency:** time it takes to process request and response.

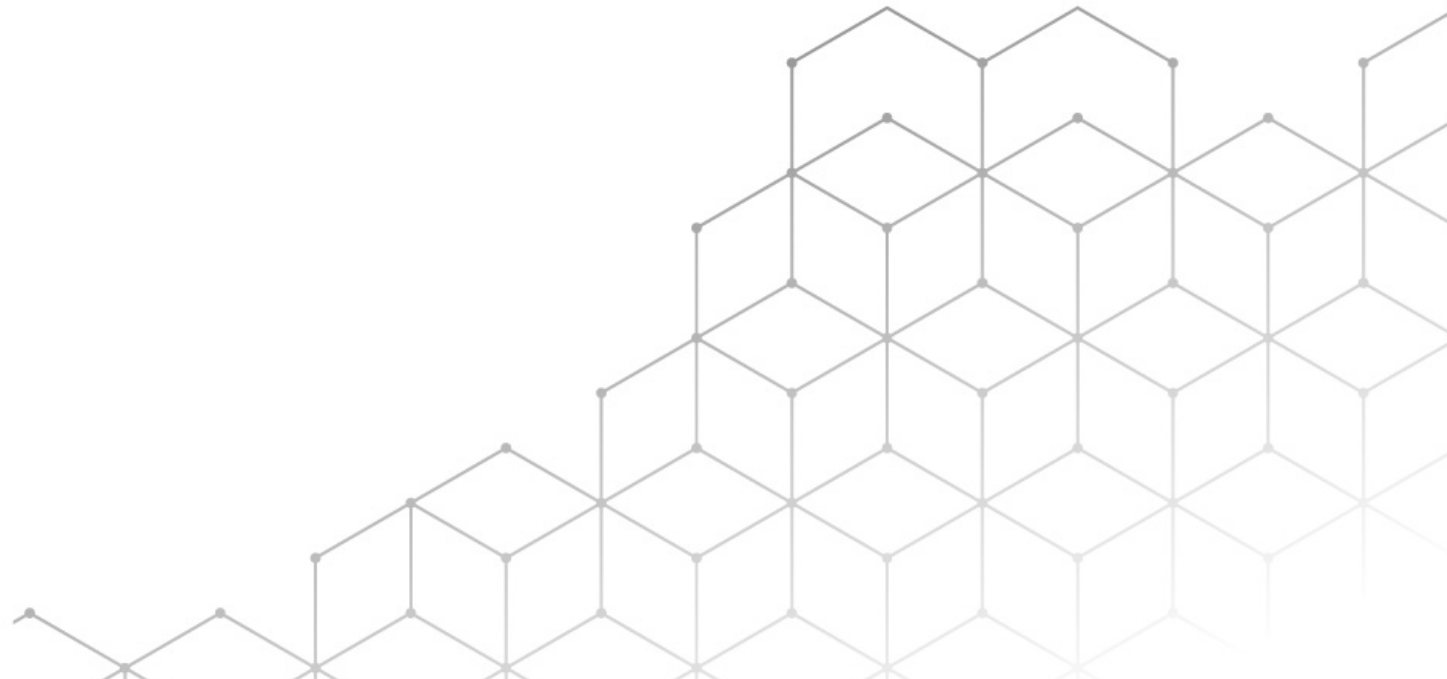
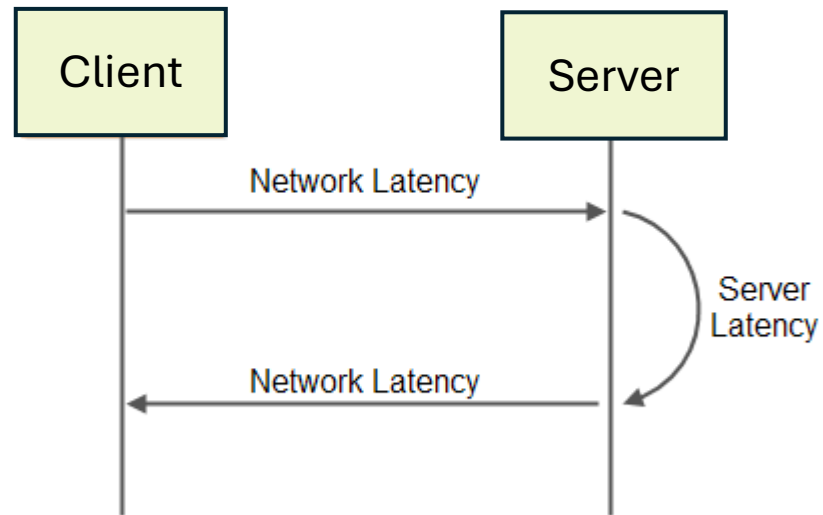
What is the typical calculation for Latency?



Latency

- Latency is a measure of time delay in a system.

network latency + server latency + network latency = 2 * network latency + server latency

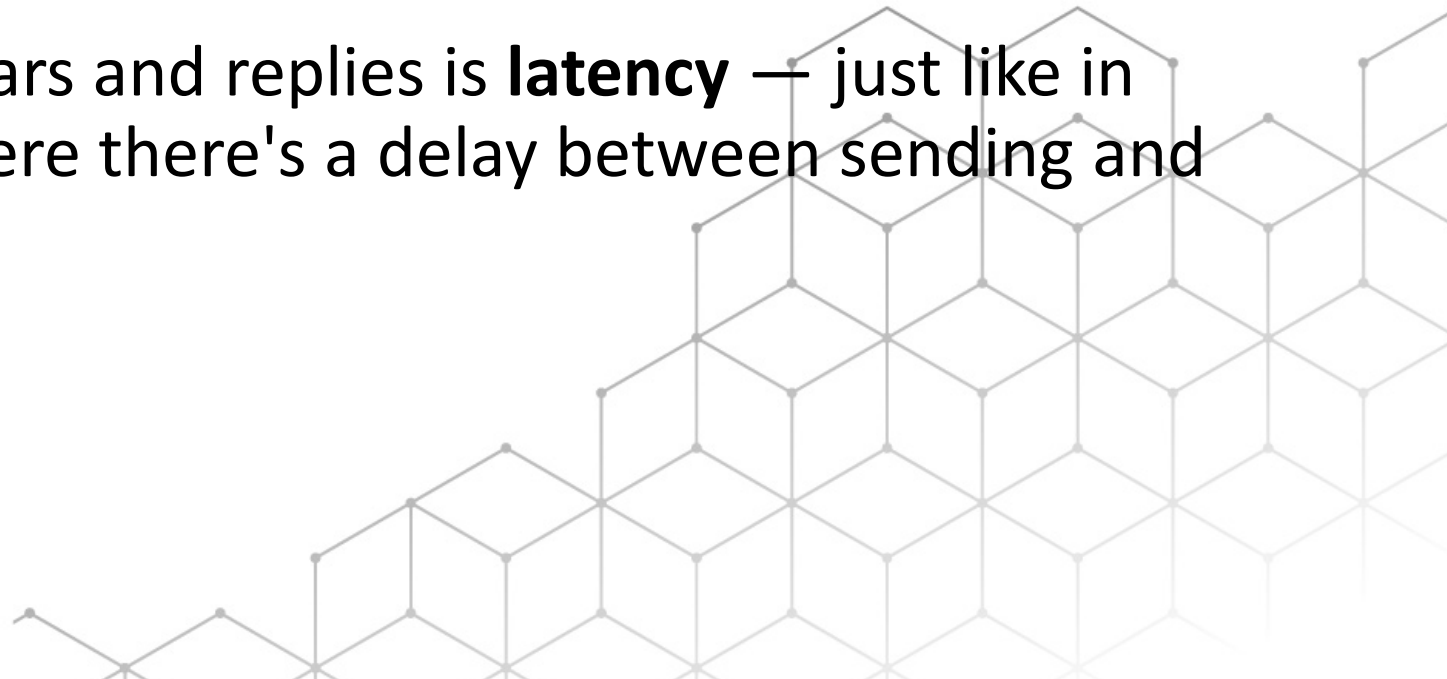


Latency Example

Imagine two people are having a conversation across a long hallway.

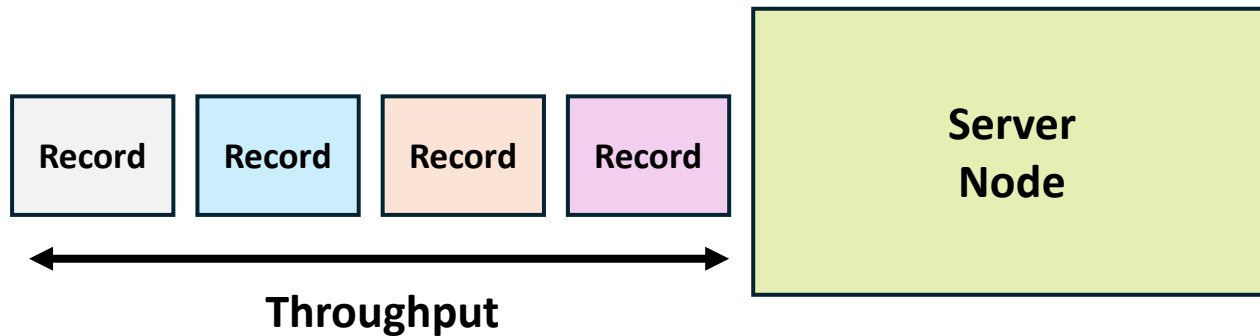
- Person A says, “Hello!”
- But Person B hears it **2 seconds later** because the sound takes time to travel.

That delay before Person B hears and replies is **latency** — just like in networks or data systems, where there's a delay between sending and receiving a message.



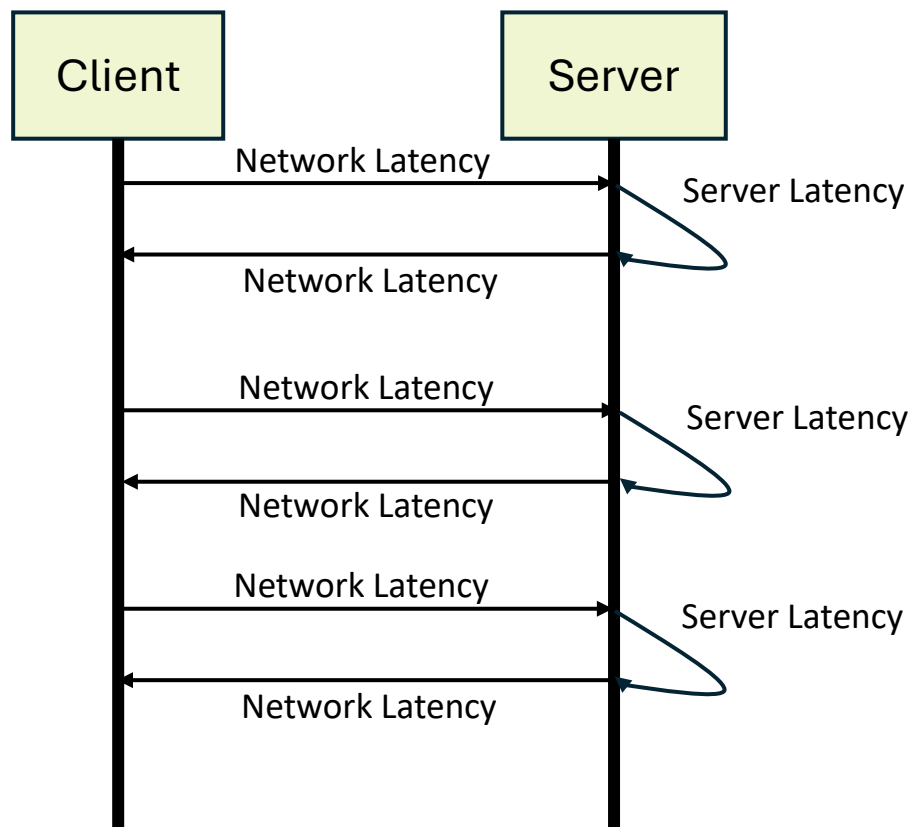
Throughput

- **Throughput** is how much work a system can do in a time interval.
- **Server throughput** is how many requests per time interval (typically per second) the server can process.

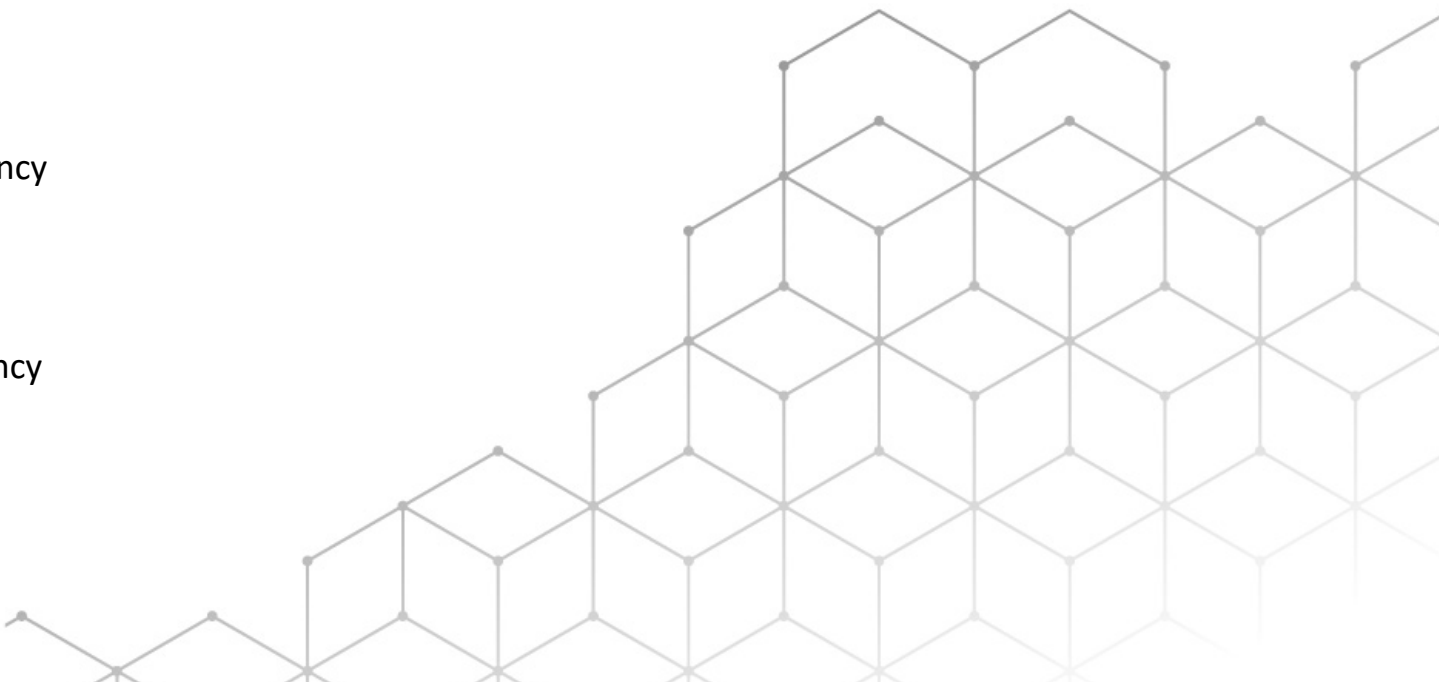


Without Batching

$$3 * (\text{network latency} + \text{server latency} + \text{network latency}) = 6 * \text{network latency} + 3 * \text{server latency}$$

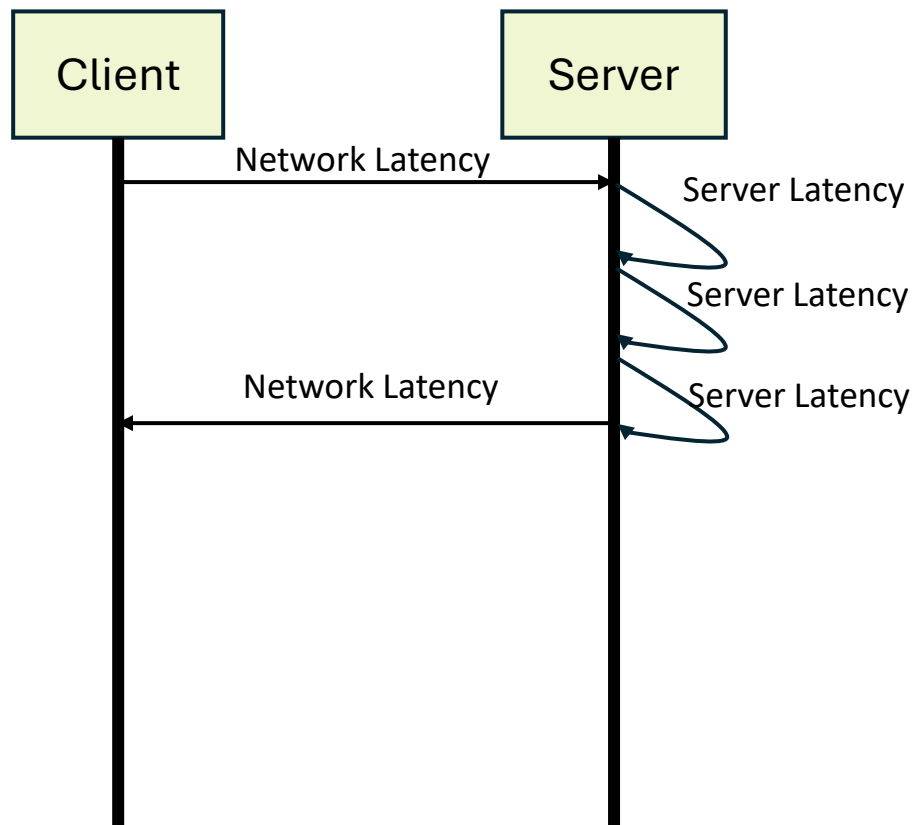


- Batching is a technique to increase the **throughput** or the **latency** of a system?



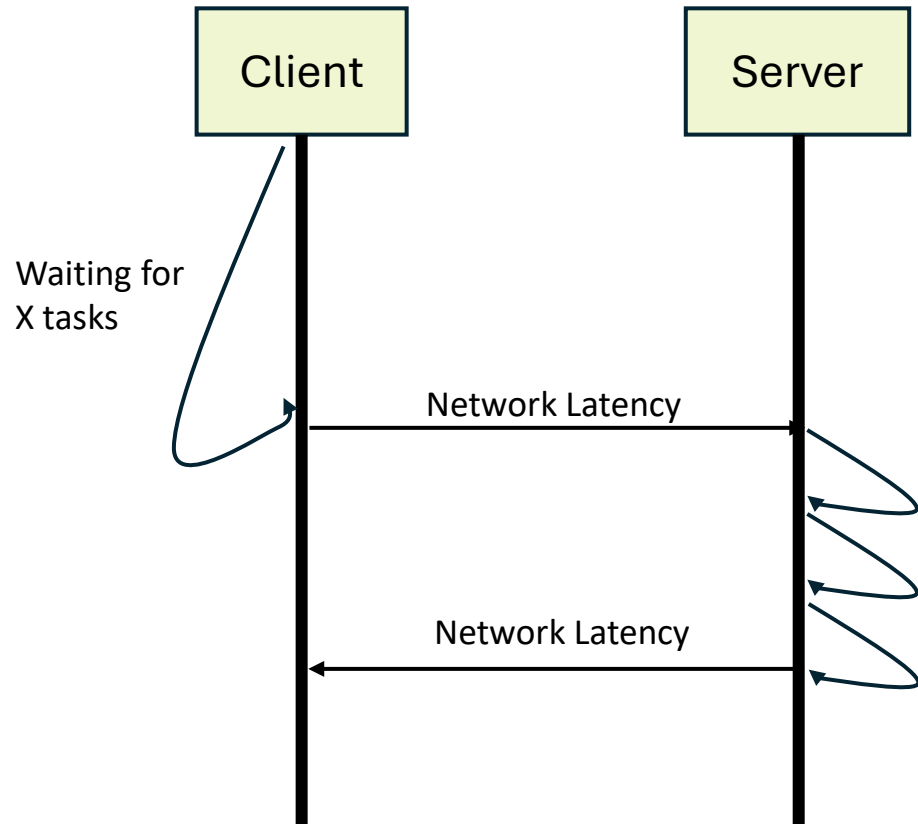
Batching

$\text{network latency} + 3 * \text{server latency} + \text{network latency} = 2 * \text{network latency} + 10 * \text{server latency}$



- Batching is a technique to increase the **throughput** of a system.
- Instead of executing each task separately, the tasks are grouped into **big batches** and executed all together.

Disadvantage of Batching



- The time it takes to collect the tasks to batch. → Disadvantage
- The first task needs to wait 2 hours before there are enough tasks to send a batch
- It will take 2 hours from the first task is collected until the client gets a response for it.

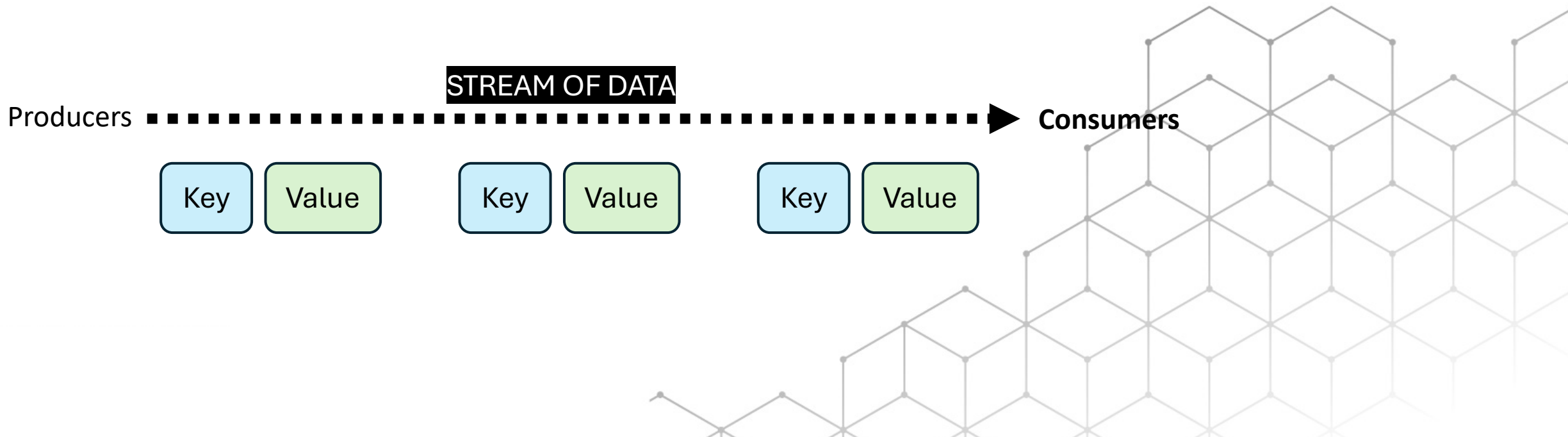
Stream Processing

- Stream processing is the process of being able to almost instantaneously analyze data that is streaming from one device to another.

Real-time Processing	Batch Processing	Stream Processing
Reacts to data in seconds or milliseconds	Processes to large volume of data all at once	Analyzes streaming in near real time
Difficult to Implement	Coding Expertise	Data output rate = Data input rate

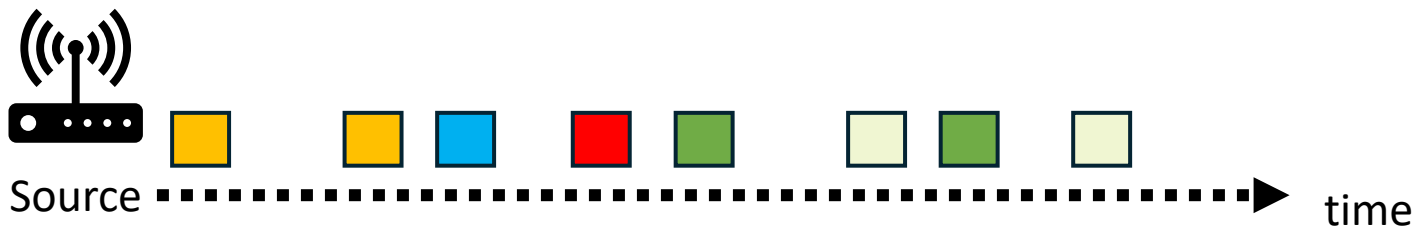
What is a Stream?

- A stream is an unbounded sequence of events that go from producers to consumers. A lot of data is produced as a stream of events, for example financial transactions, sensor measurements, or server logs.



Unbound Data

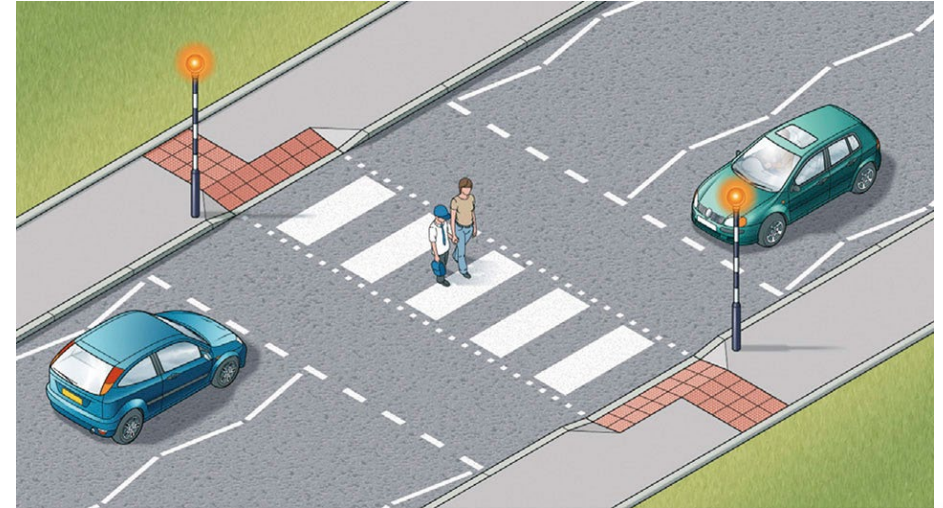
- Unbound data is unpredictable, infinite, and **not always sequential**. The data creation is a never ending cycle, it just keeps going and going.



Real world = Unbound Data

How much unbound data (stimuli) did I process and analyze?

- Watching for cars in the parking lot and calculating where and when to walk
- Knowing the location of my car and path to get to car
- Puddles, pot holes, and pedestrians to navigate



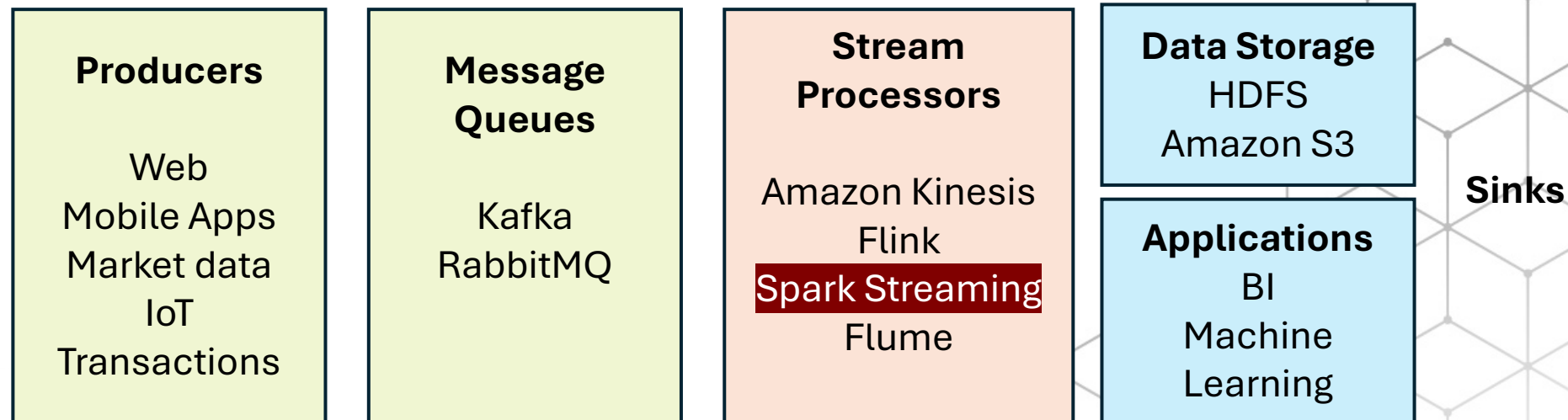
Applications of Stream Processing

- Healthcare: continuous monitoring of sensor data
- Smart cities: traffic patterns and congestion management
- Manufacturing: optimization and predictive maintenance
- Cyber security, anomaly detection: web or network log processing
- Financial: stock market time series
- Advertising: location- or action-based advertising



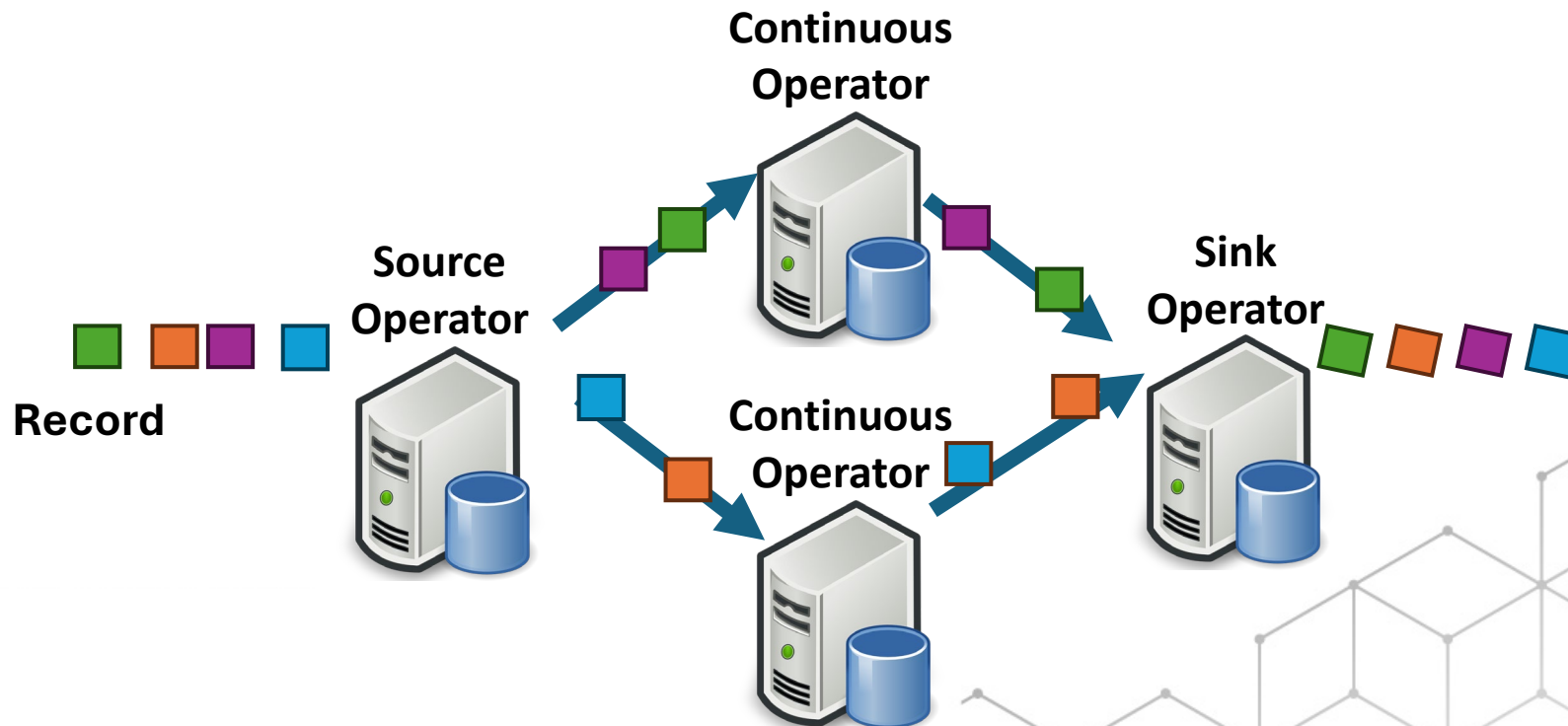
Architecture of Stream Processing

- Producers: Generate data (e.g., log files, sensors).
- Message Queues: Temporarily store incoming streams
- Stream Processors: Process data sequentially or in micro-batches
- Sinks/Consumers: Store or act on processed data



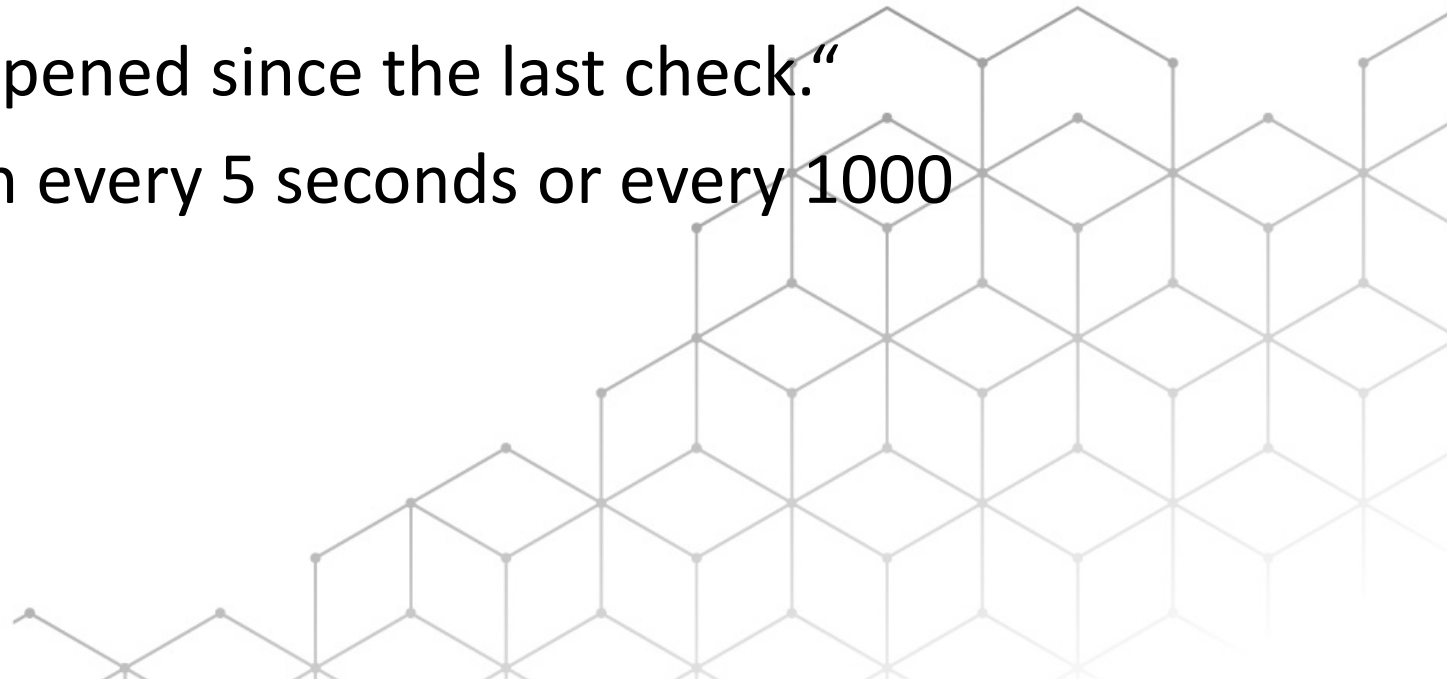
Real-Time Stream Processor

- Most stream processors work with a continuous operator model
- Each operator processes the stream one record at a time and forwards the records to other operators in the pipeline



Discrete Operator Model

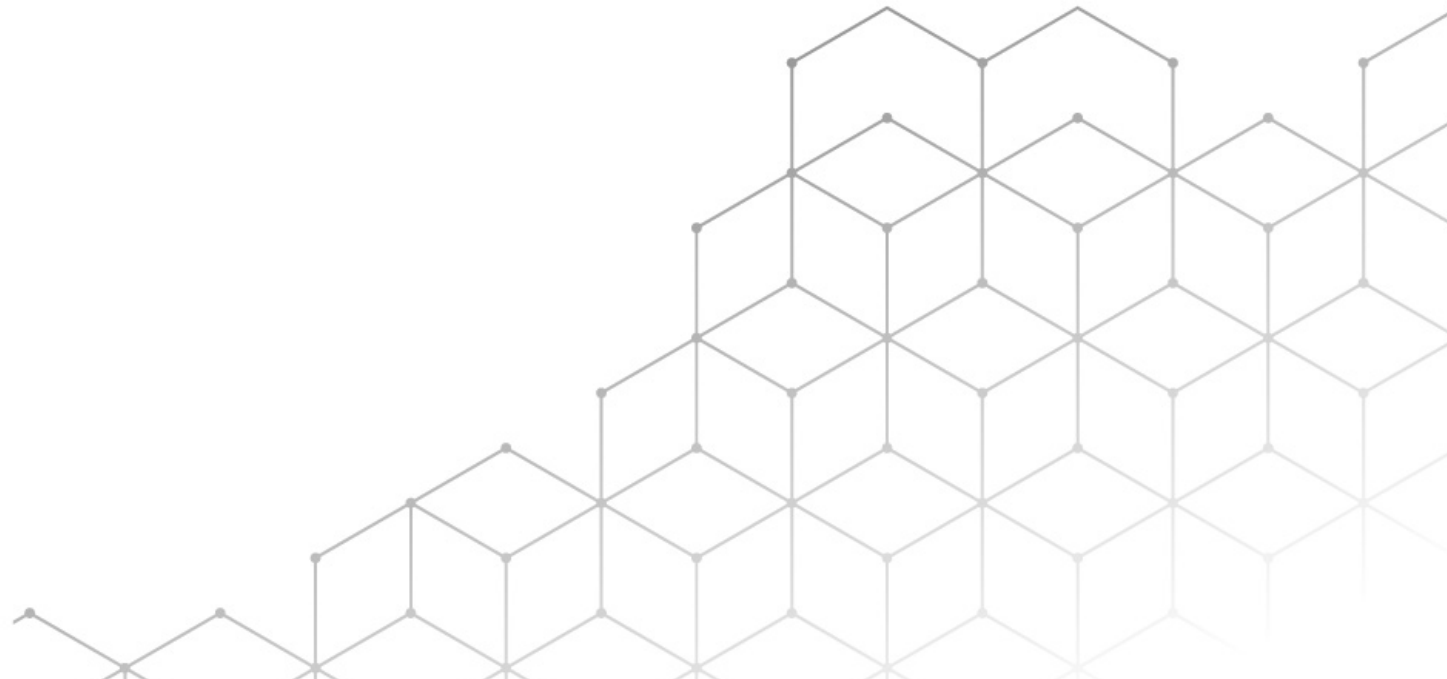
- The **alternative** to a **continuous operator model** is usually a **discrete operator model**.
- Processes data in chunks, windows, or batches.
- Thinks in terms of "what happened since the last check."
- Example: updating prediction every 5 seconds or every 1000 messages.



Example of Raindrops

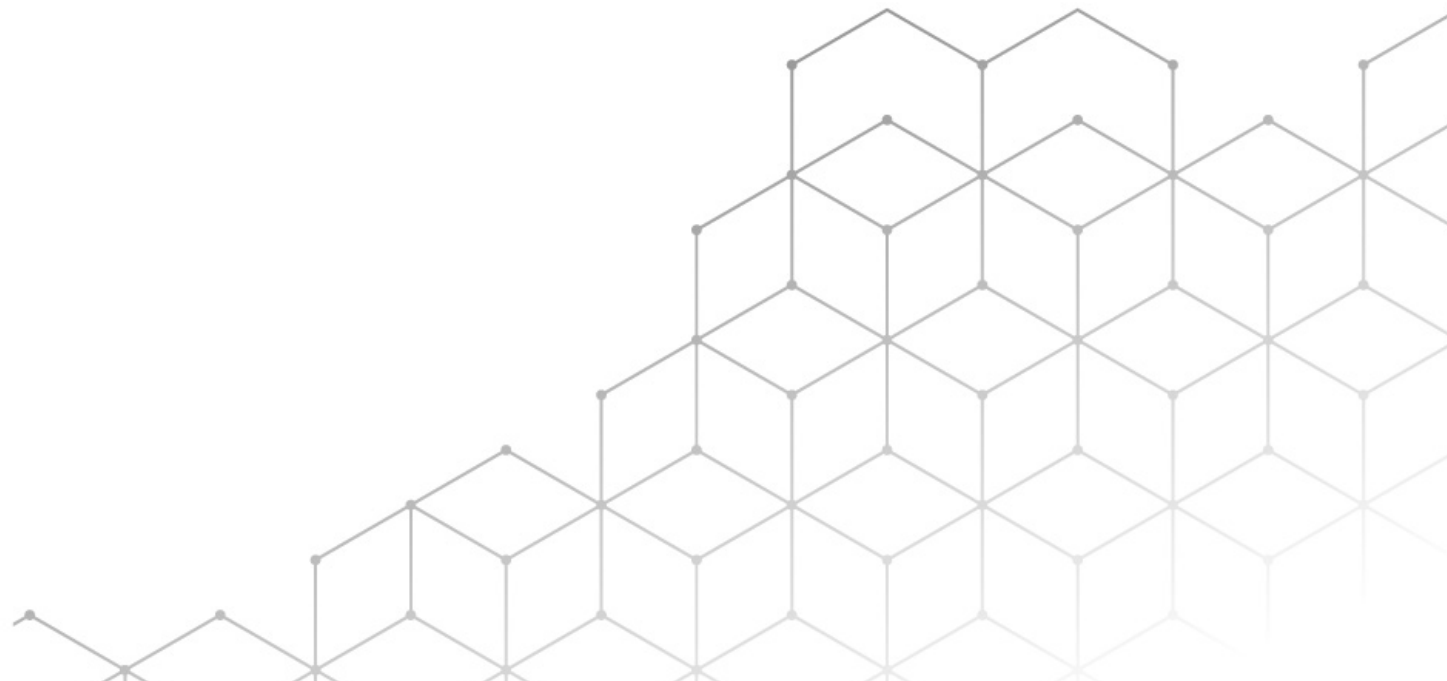
If data were raindrops:

- **Continuous:** you're measuring the water *height* at every instant.
- **Discrete:** you're measuring how much water you collected *every 10 minutes*.



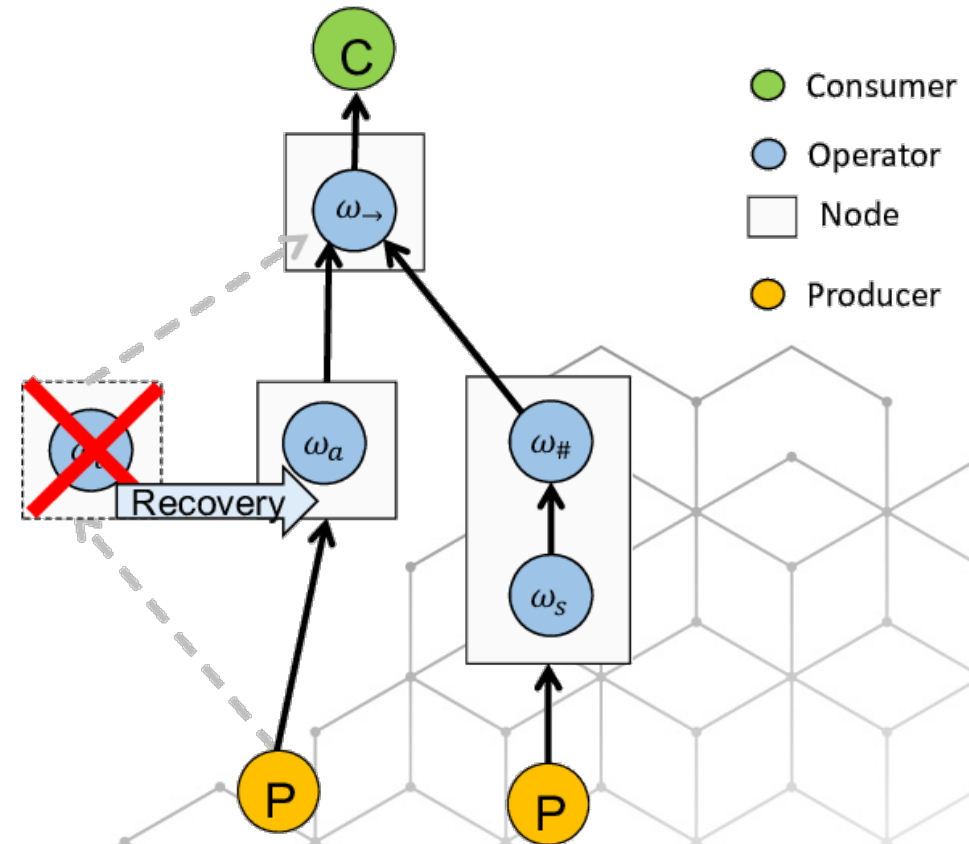
Problems to Solve in Stream Processors

- Fault Tolerance
- Load Balancing
- Unification of Batch, Stream Workloads
- Data Analytics



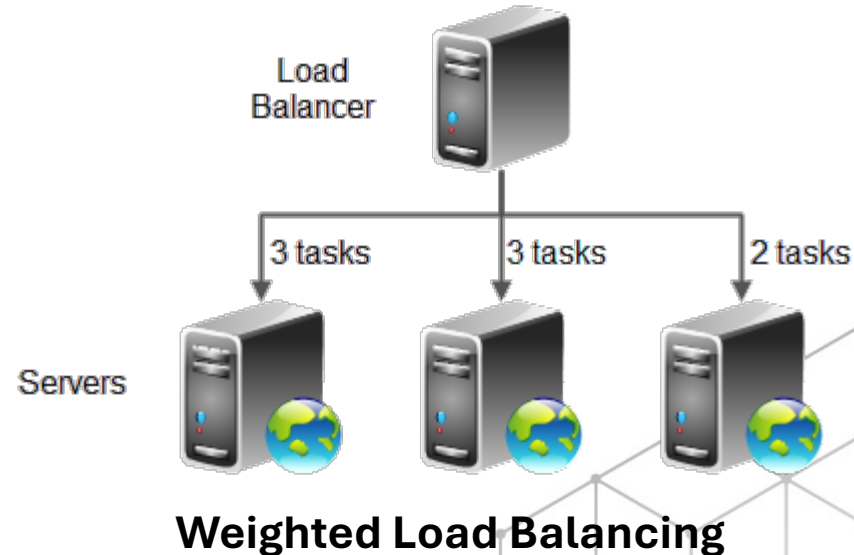
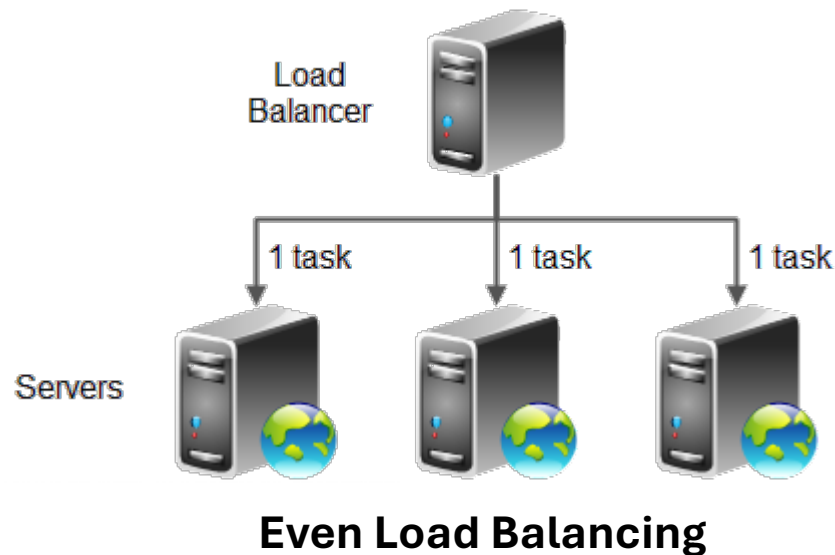
Fault Tolerance

- **Fault Tolerance** - With greater scale, there is a higher likelihood of a cluster node failing or unpredictably slowing down (i.e. **stragglers**).
- The static allocation of continuous operators to worker nodes makes it challenging for traditional systems to recover quickly from faults and stragglers.



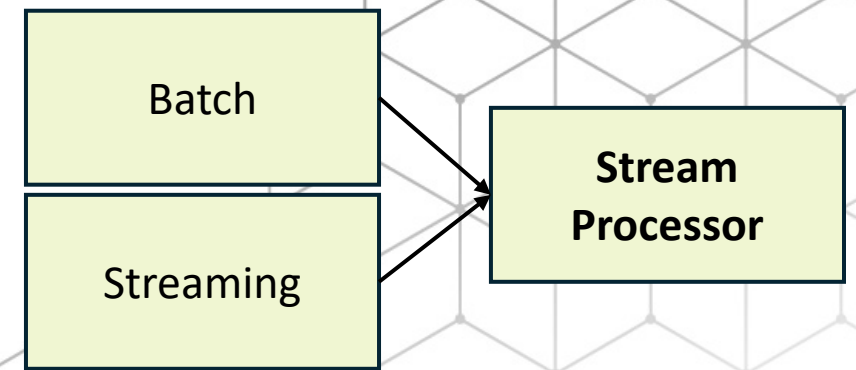
Load Balancing

- **Load balancing** - Uneven allocation of the processing load between the workers can cause bottlenecks. The system needs to dynamically adapt the resource allocation based on the workload.



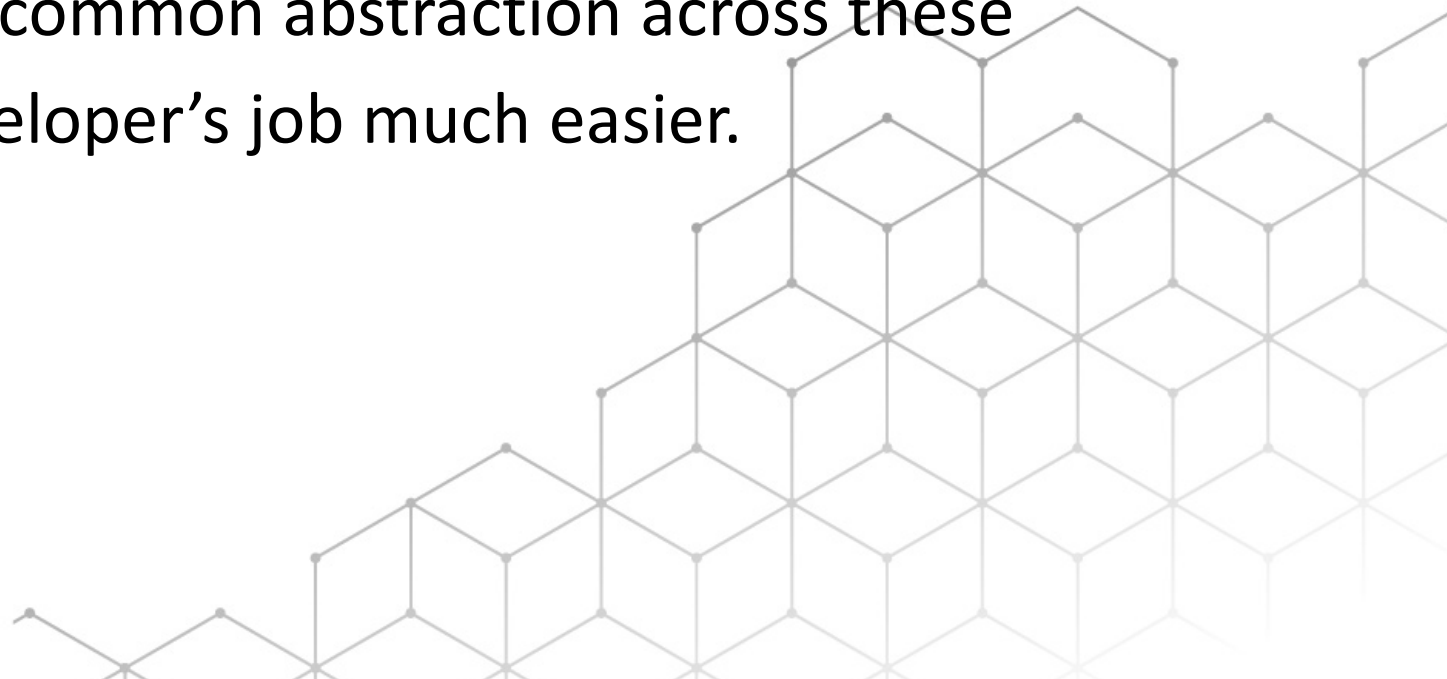
Unification of Workloads

- **Unification of streaming, batch and interactive workloads** - In many use cases, it is also attractive to query the streaming data interactively (after all, the streaming system has it all in memory), or to combine it with static datasets. This requires a single engine that can combine batch, streaming and interactive queries.



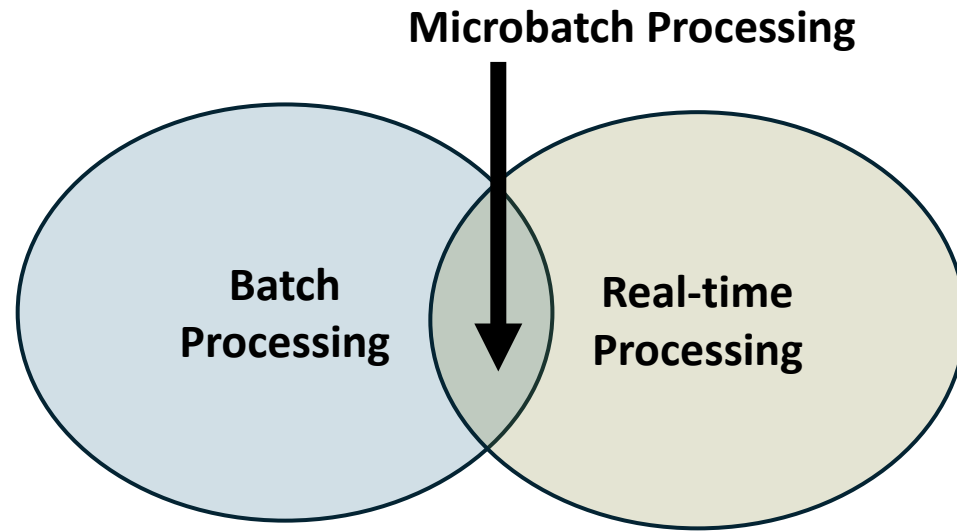
Data Analytics

- **Advanced analytics like machine learning and SQL queries** - More complex workloads require continuously learning and updating data models, or even querying the “latest” view of streaming data with SQL queries. Again, having a common abstraction across these analytic tasks makes the developer’s job much easier.



Micro Batch Processing

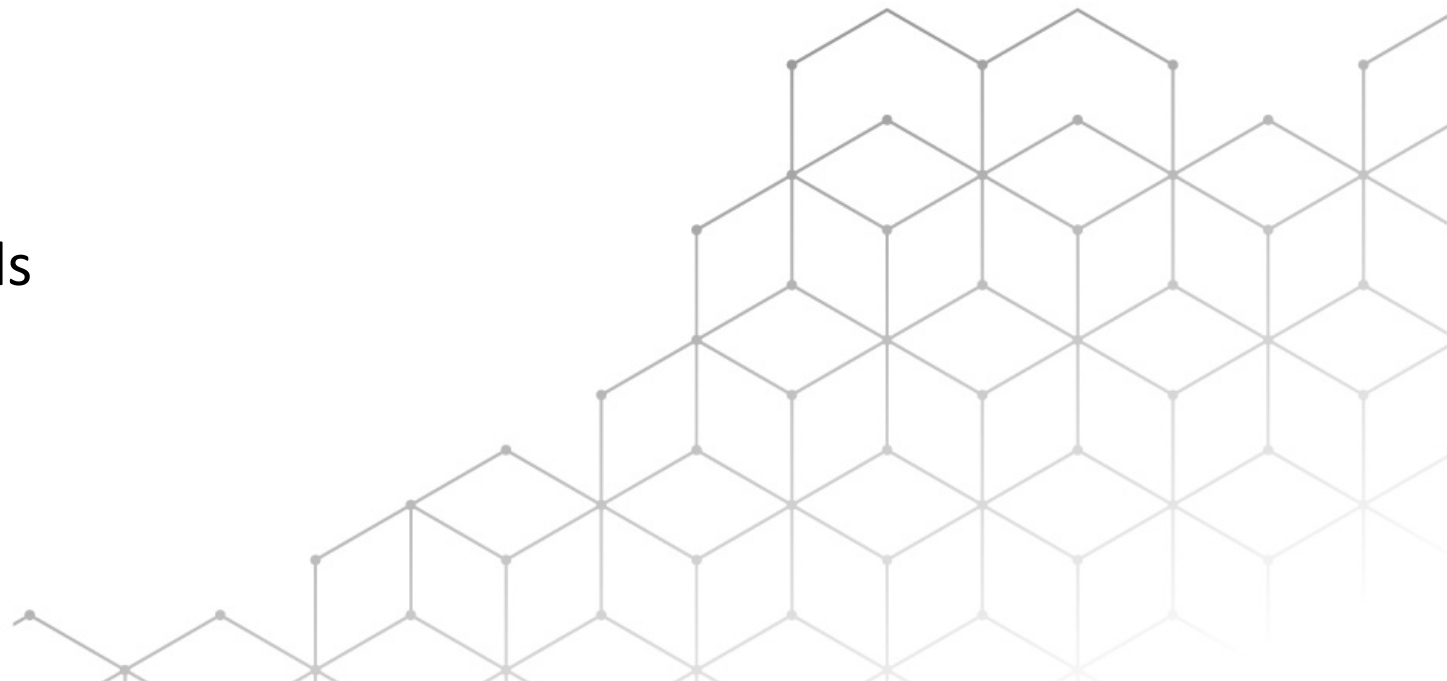
- Microbatch processing is useful when we need very fresh data, but not necessarily real-time – meaning we can't wait an hour or a day for a batch processing to run, but we also don't need to know what happened in the last few seconds.



Agenda for Today

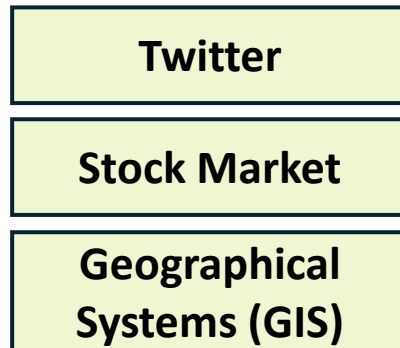


- Data Streaming
- Spark Streaming: talk about a specific platform for streaming
 - Why Spark Streaming?
 - What is Spark Streaming?
 - Overview of Spark Streaming
 - Spark Streaming Features
 - Spark Streaming Workflow
 - Spark Streaming Fundamentals
 - Fault-Tolerance
- D-stream Abstraction



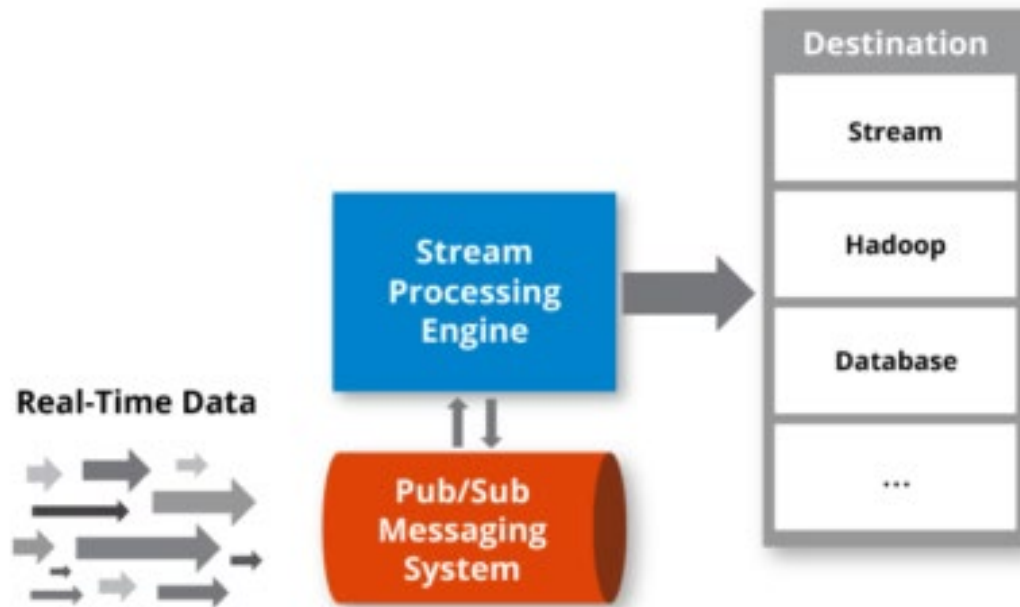
Why Spark Streaming?

- We can use Spark Streaming to stream (near) real-time data from various sources like Twitter, Stock Market and Geographical Systems and perform powerful analytics to help businesses.



Why Near Real-Time Streaming?

- **Processing each event instantly**
- **Micro-batches** — small chunks of data (every 1 or 2 seconds),

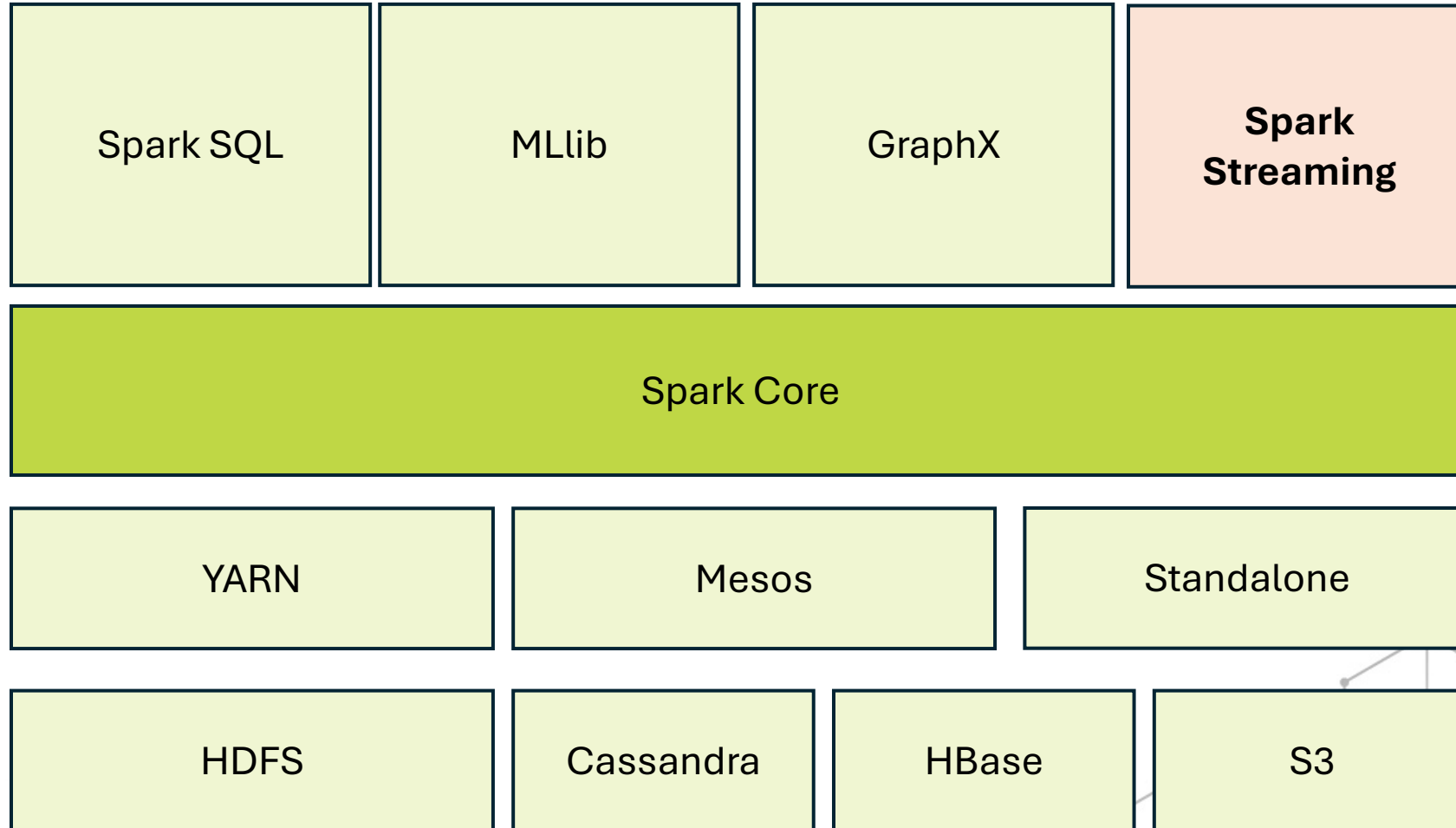


What is Spark Streaming?

- Spark Streaming integrates with Spark Core and other Spark components for real-time data processing. Provides high throughput and fault tolerance for streaming data.

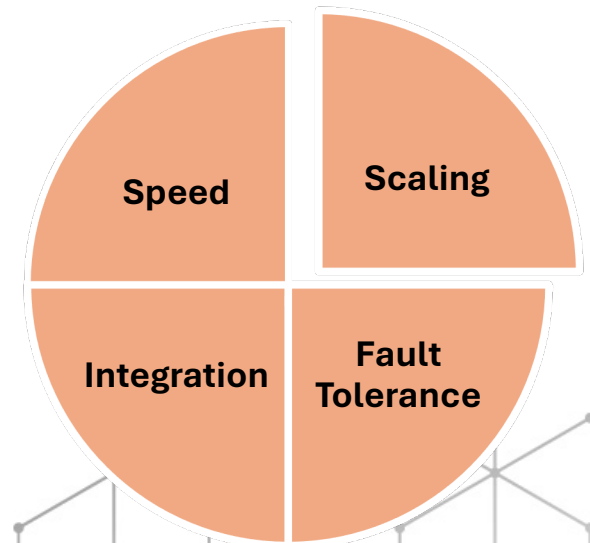


Spark Streaming in the Architecture

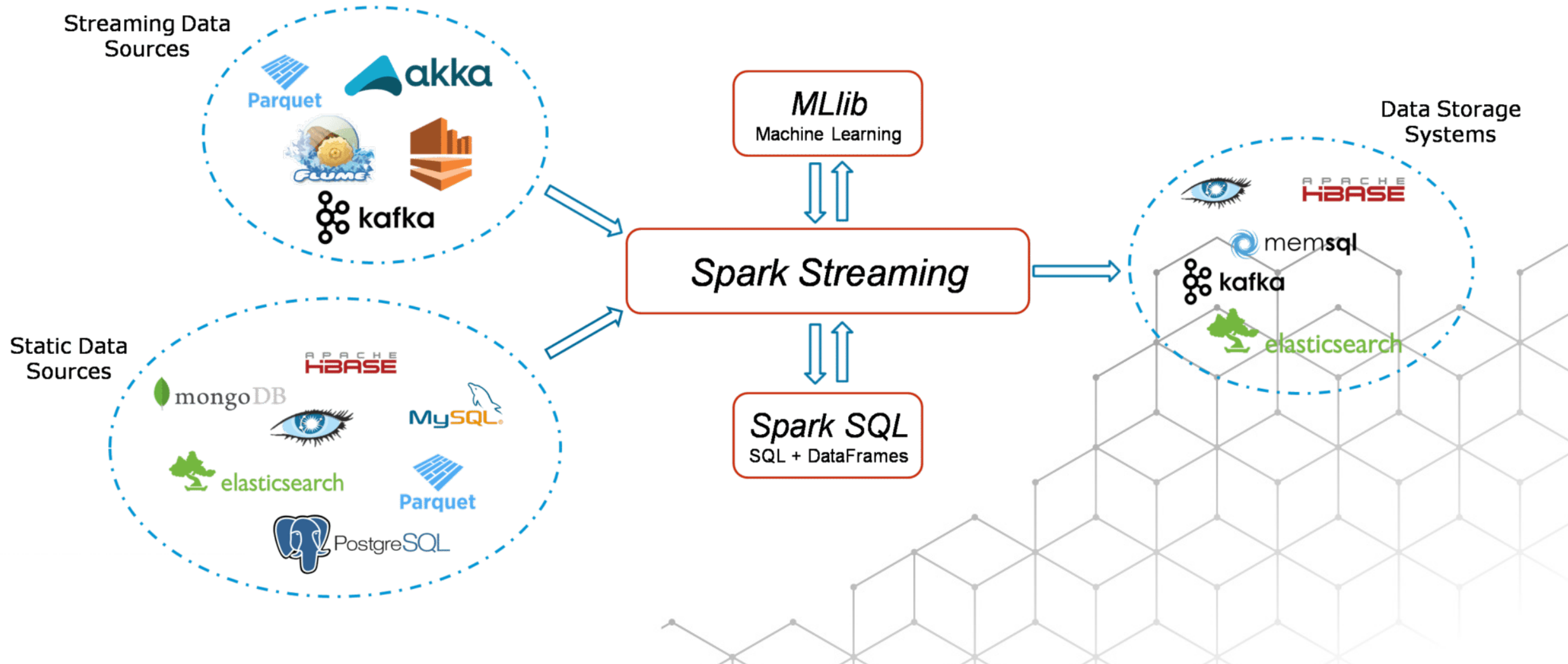


Spark Streaming Features

- **Scaling:** Spark Streaming can easily scale to hundreds of nodes.
- **Speed:** It achieves low latency.
- **Fault Tolerance:** Spark can recover from failures.
- **Integration:** Spark integrates with batch and real-time processing.

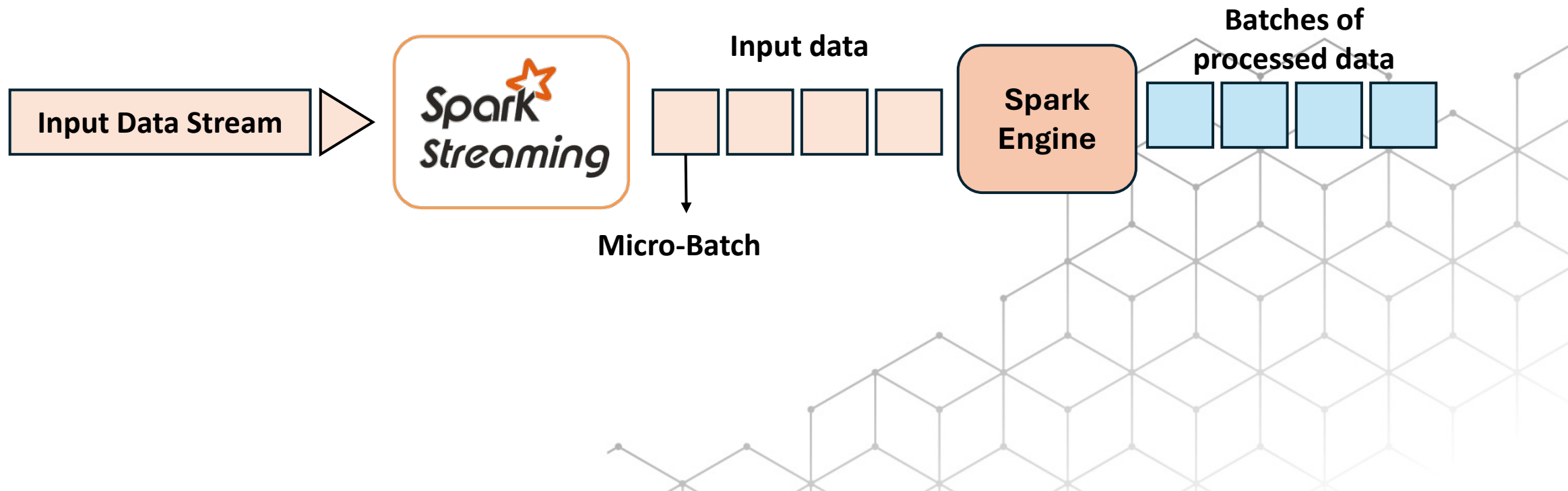


Spark Streaming Workflow



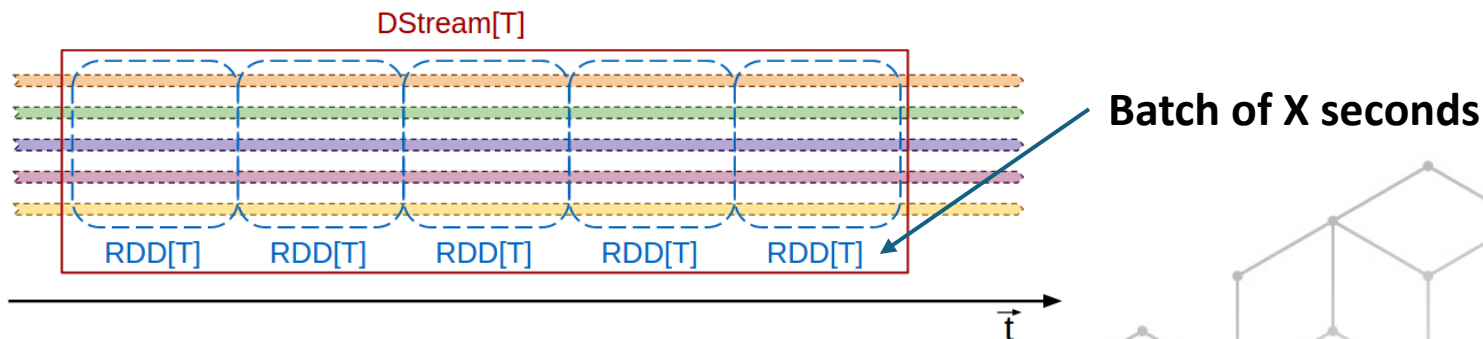
Overview of Spark Streaming

- It is a useful addition to the **core Spark API**. Spark Streaming enables high-throughput and fault-tolerant stream processing of live data streams.



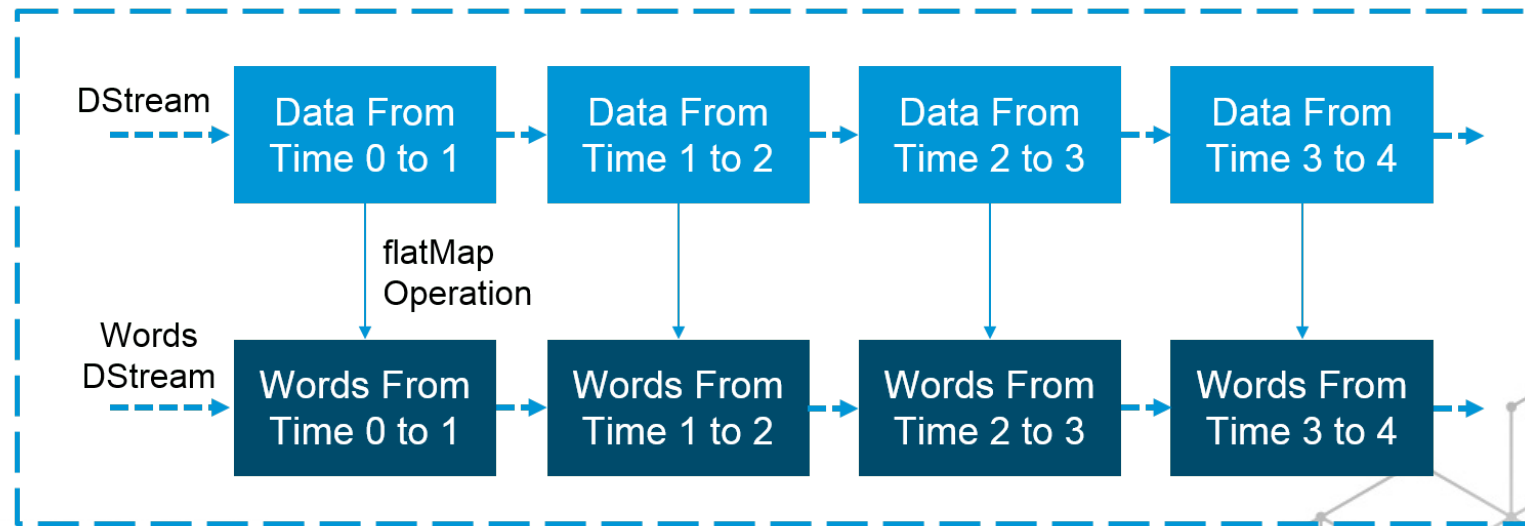
Discretized (Micro Batching) Processing

- Run a streaming computation as a series of very small, deterministic batch jobs
- Chop up the live stream into batches of X seconds
- Spark treats each batch of data as RDDs and processes them using RDD operations
- Finally, the processed results of the RDD operations are returned in batches



DStream

- Discretized Stream is the basic abstraction of Spark Streaming
- It is a continuous stream of data
- It is received from a **data source** or a **processed data stream**

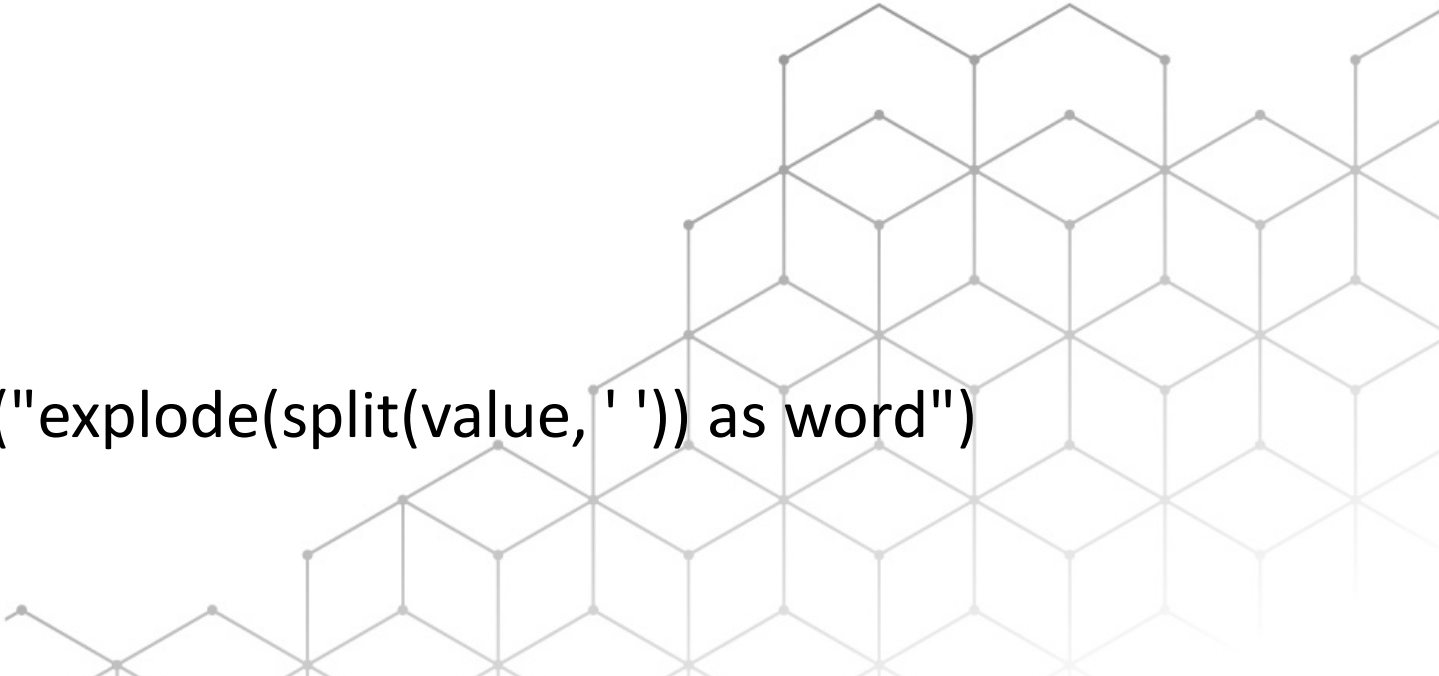


Extracting words from an Input DStream

Code

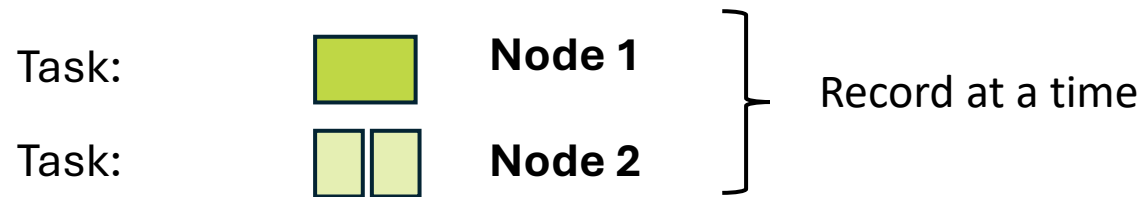
```
# Create a streaming DataFrame from a socket
# (For testing: you can send messages to port 9999 using 'nc -lk 9999')
streaming_df = spark.readStream \
    .format("socket") \
    .option("host", "localhost") \
    .option("port", 9999) \
    .load()

# Split the lines into words
words = streaming_df.selectExpr("explode(split(value, ' ')) as word")
```



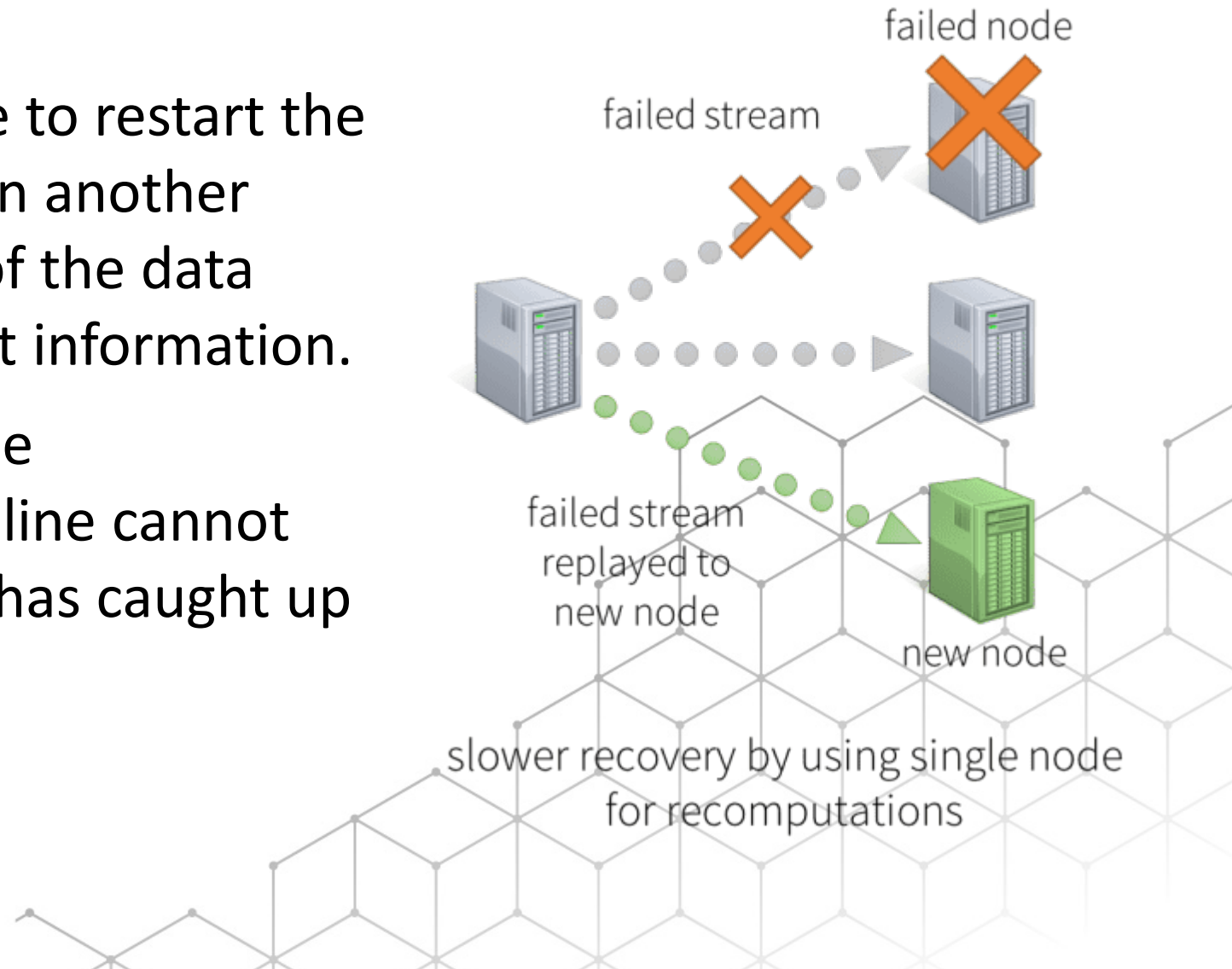
Load Balancing

- In the traditional record-at-a-time approach, if one of the tasks is more computationally intensive, the node statically assigned to process that task will become a bottleneck and slow down pipeline.
- In Spark Streaming, the tasks will be load balanced -- some workers will process a few longer tasks, others more of the shorter tasks.



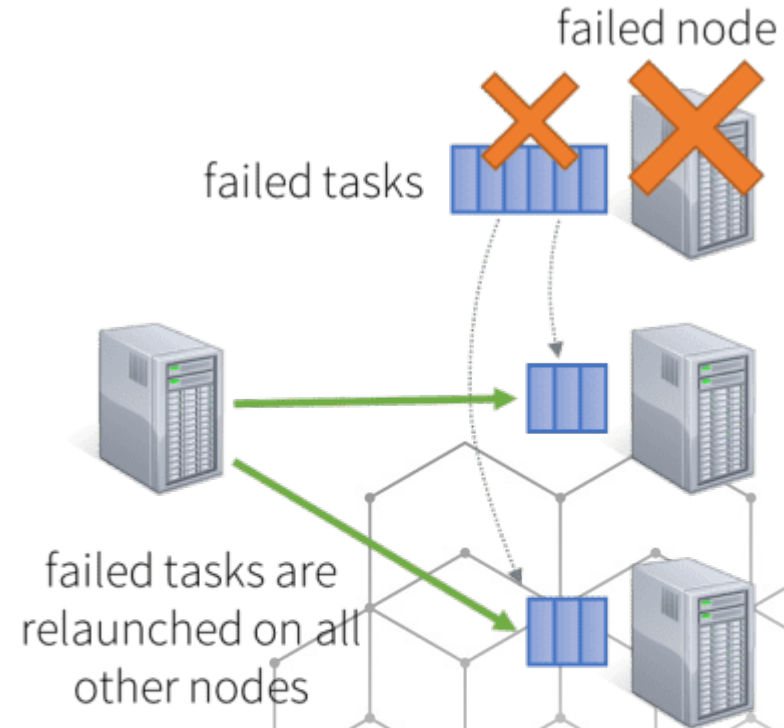
Conventional Fault Tolerance

- In case of node failures, have to restart the failed continuous operator on another node and replay some part of the data stream to recompute the lost information.
- Only one node is handling the recomputation, and the pipeline cannot proceed until the new node has caught up after the replay.



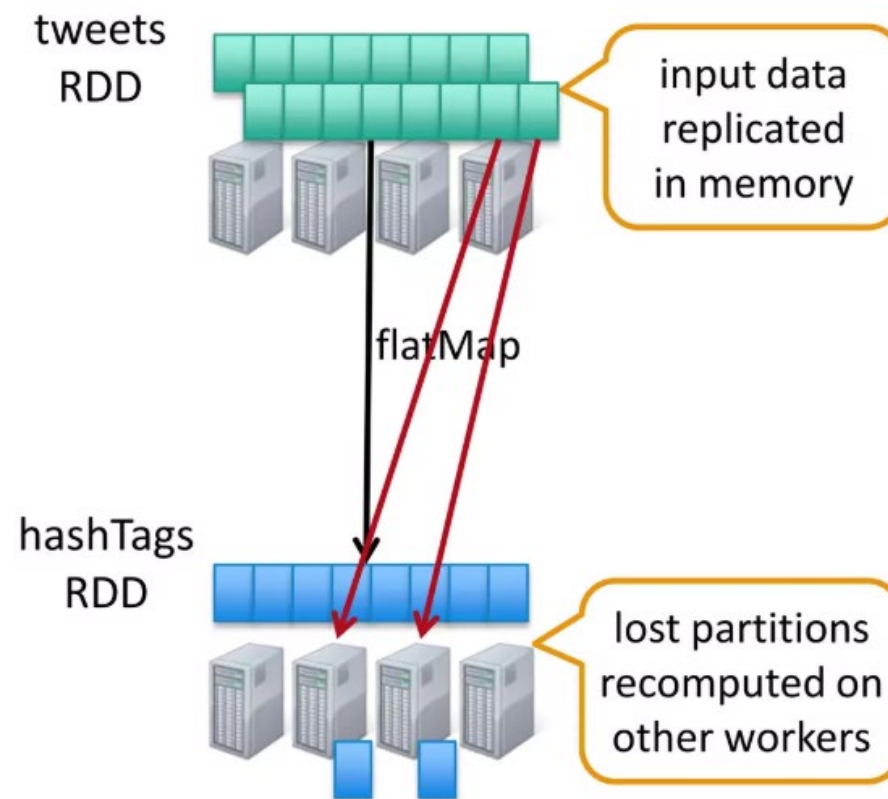
Spark Streaming Fault Tolerance

- In Spark, the computation is already discretized into small, deterministic tasks that can **run anywhere** without affecting correctness.
- Failed tasks can be relaunched in parallel on all the other nodes in the cluster, thus evenly distributing all the recomputations across many nodes, and recovering from the failure faster than the traditional approach.



Replication and Fault Tolerance

- Batches (Micro) of input data are replicated in memory for fault-tolerance
- Data lost due to worker failure can be recomputed from replicated input data
- Therefore, all transformed data is fault-tolerant



Example – Get Hashtags from Twitter (1)

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)
```

- **DStream:** A sequence of distributed datasets (RDDs) representing a distributed stream of data.

Twitter Streaming API:

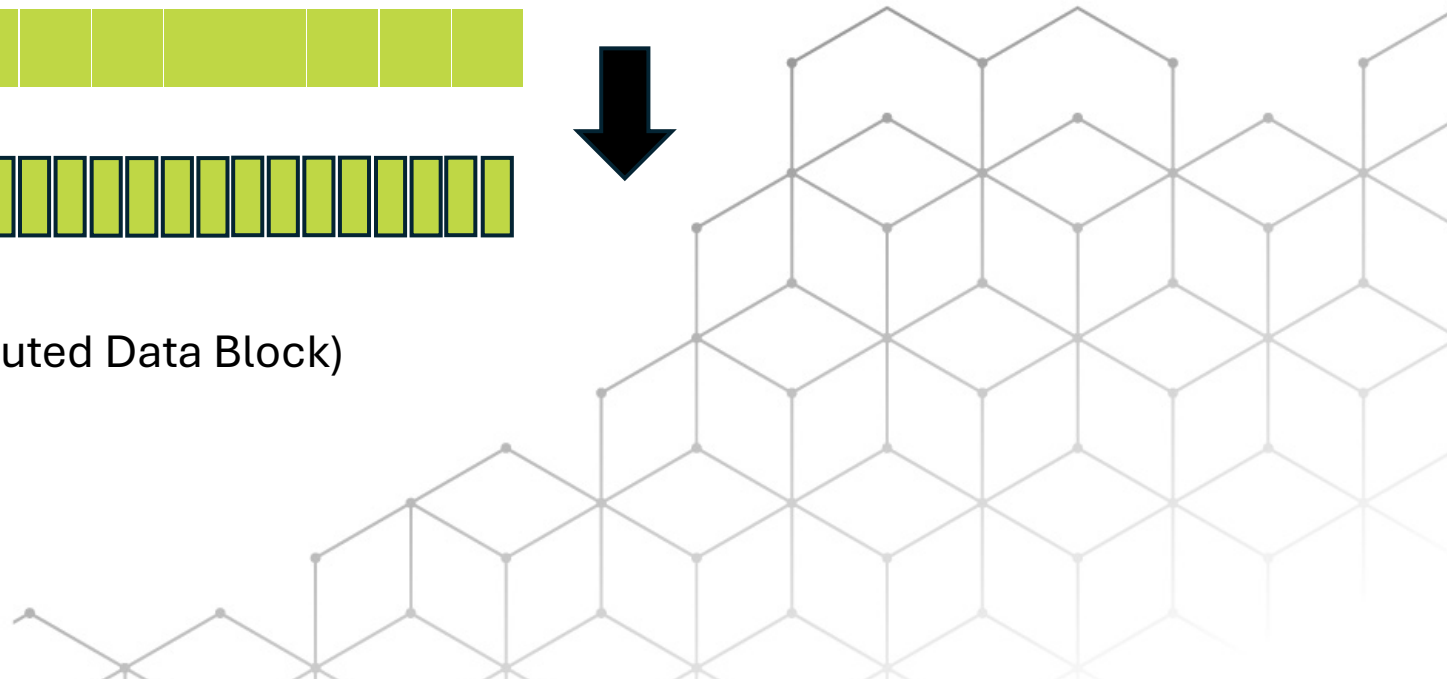


Tweets DStream:



RDD

(Immutable, Distributed Data Block)



Example – Get Hashtags from Twitter (2)

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)  
val hashTags = tweets.flatMap(status => getStatus(status))
```

- **Transformation:** Transform data in one stream to create another

Twitter Streaming API:



Tweets DStream:

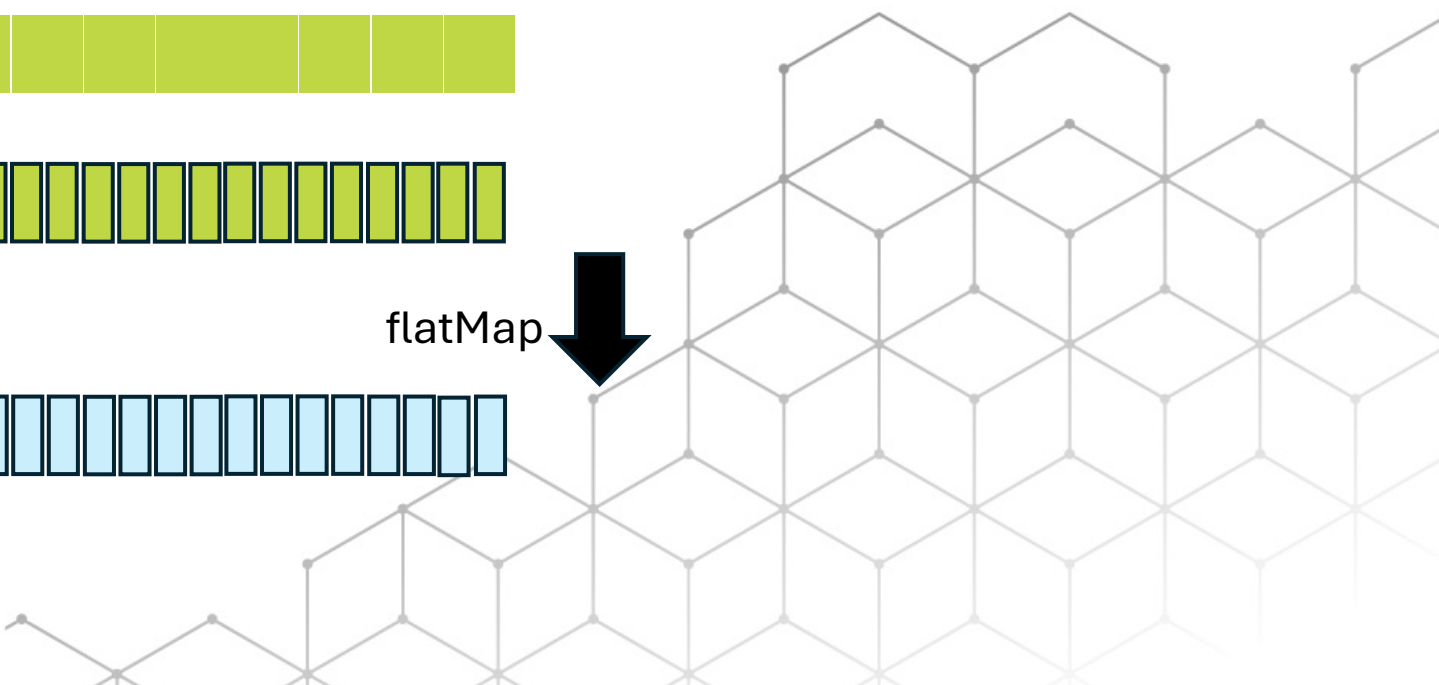


HashTags DStream:



New
RDD

flatMap



Example – Get Hashtags from Twitter (2)

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)  
val hashTags = tweets.flatMap(status => getStatus(status))  
hashTags.saveAsHadoopFiles("hdfs:\\a.txt")
```

- Save results to external storage (e.g. HDFS)

Twitter Streaming API:



Tweets DStream:



HashTags DStream:



HDFS:



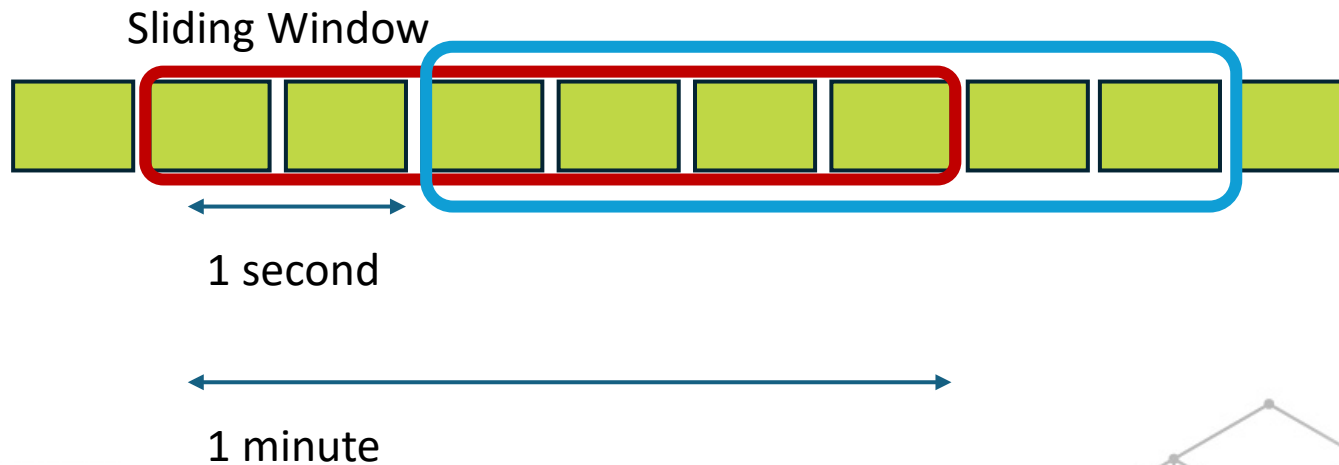
Example – Count Hashtags Last Min (1)

```
val tweets = ssc.twitterStream(<Twitter username>, <Twitter password>)  
val hashTags = tweets.flatMap(status => getStatus(status))  
Val tagsCounts = hashTags.window(Minutes(1), Seconds(1)).countByValue()
```

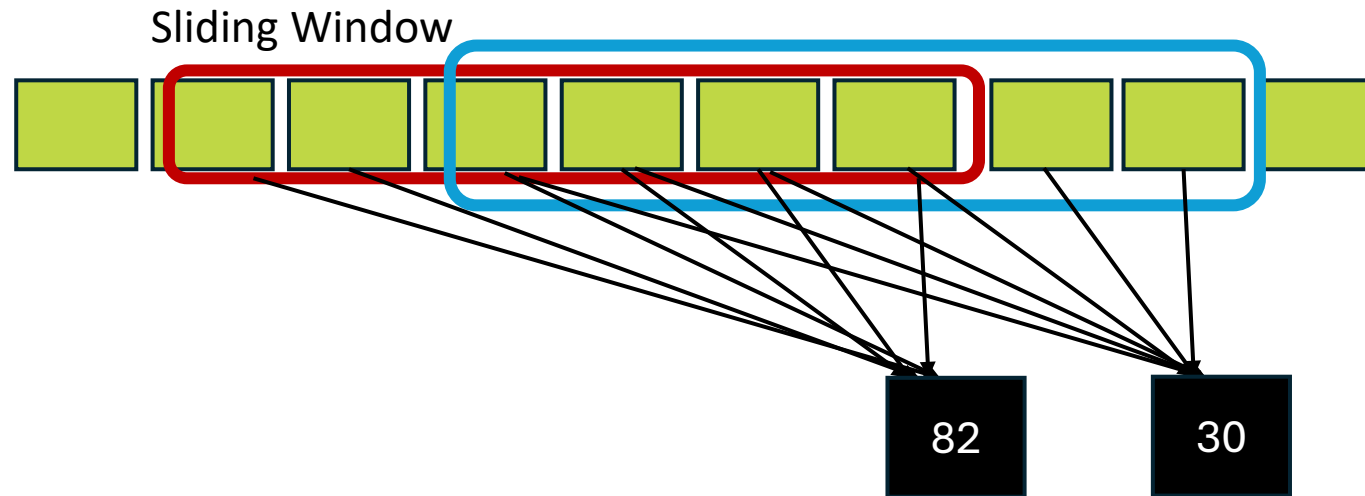
Sliding Window

Sliding Window
Size

Sliding Window
Interval



Example – Count Hashtags Last Min (2)



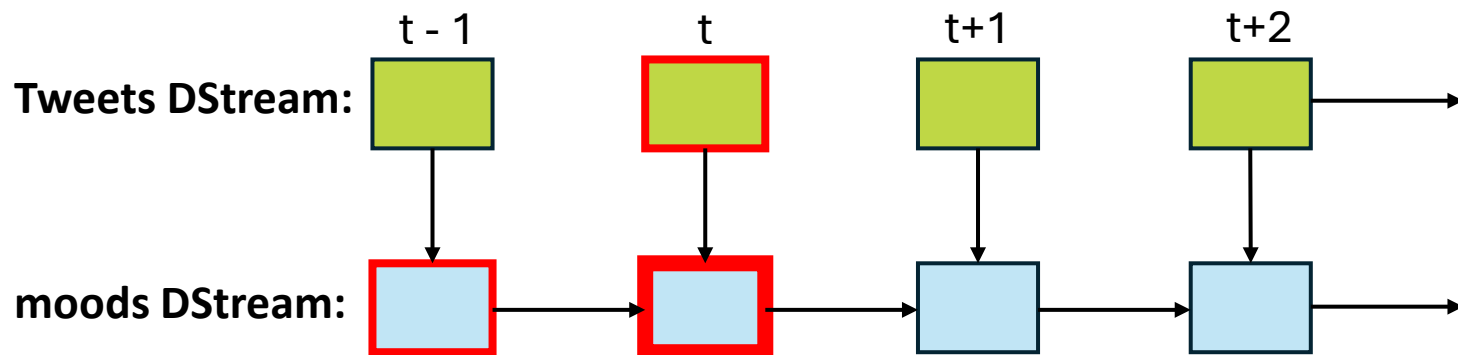
Key Concepts

- **DStream** – sequence of RDDs representing a stream of data
 - Twitter, HDFS, Kafka, Flume, ZeroMQ, Akka Actor, TCP sockets
- **Transformations** – modify data from one DStream to another
 - Standard RDD operations – map, countByValue, reduceByKey, join, ...
 - Stateful operations – window, countByValueAndWindow, ...
- **Output Operations** – send data to external entity
 - saveAsHadoopFiles
 - saves to HDFS
 - foreach do anything with each batch of results

Arbitrary Stateful Computations

- Maintain arbitrary state
 - Maintain per-user mood as state, and update it with his/her tweets

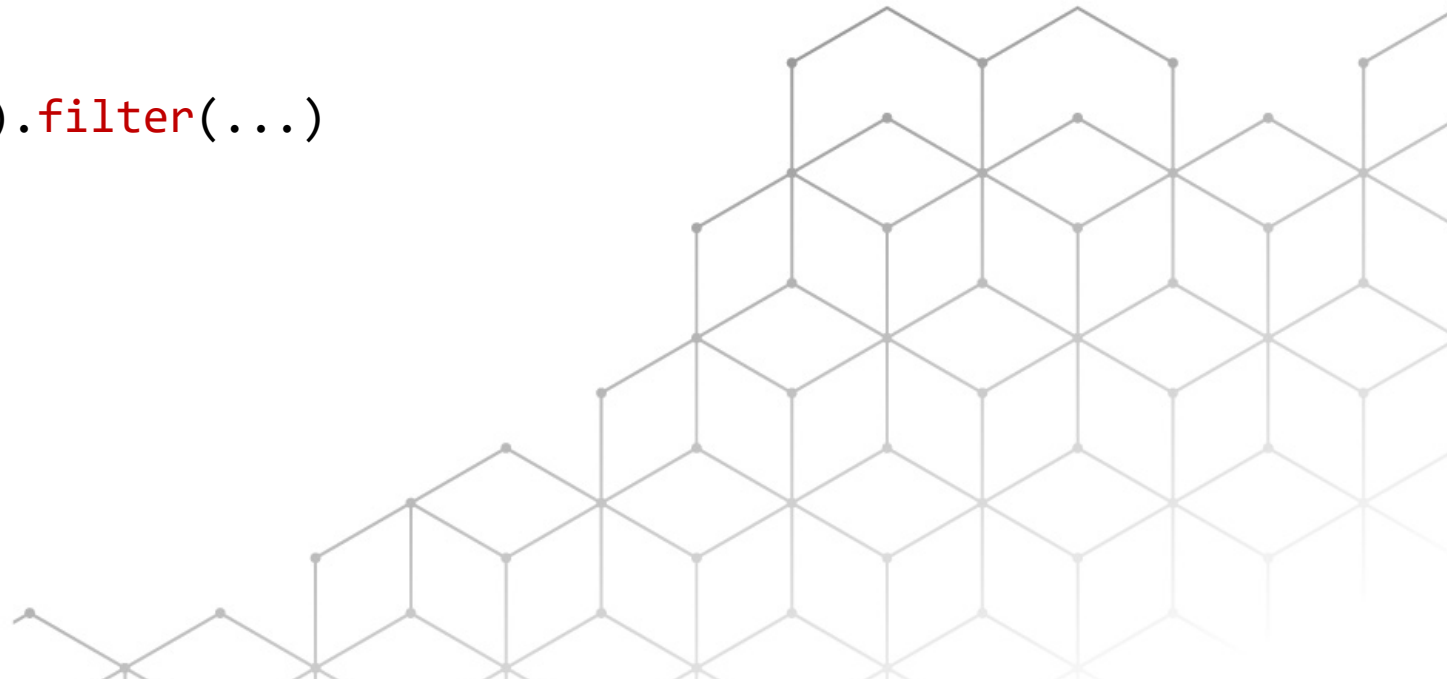
```
moods = tweets.updateStateByKey(tweet => updateMood(tweet))  
updateMood(newTweets, lastMood) => newMood
```



Combine Batch and Stream Processing

- Do arbitrary Spark RDD computation within Dstream
- Join incoming tweets with a spam file to filter out bad tweets

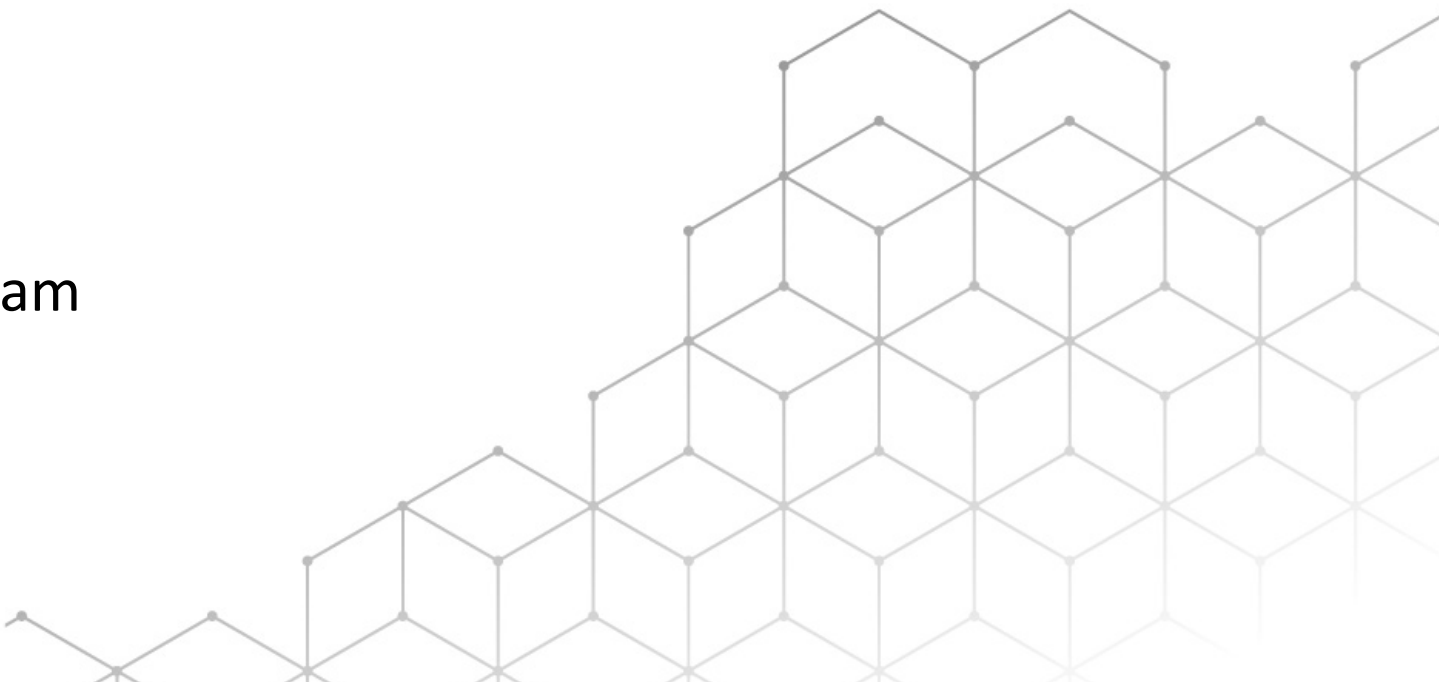
```
tweets.transform(tweetsRDD => {  
    tweetsRDD.join(spamHDFSFile).filter(...)  
})
```



Agenda for Today

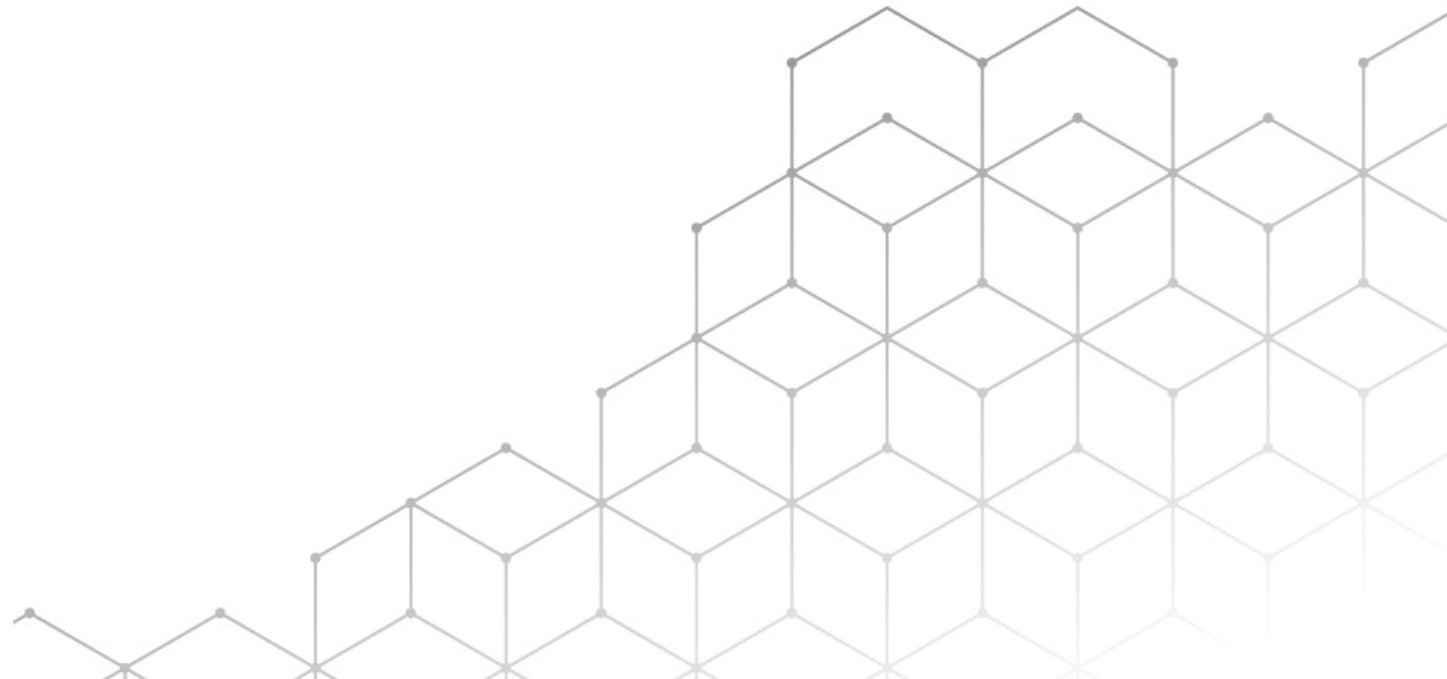
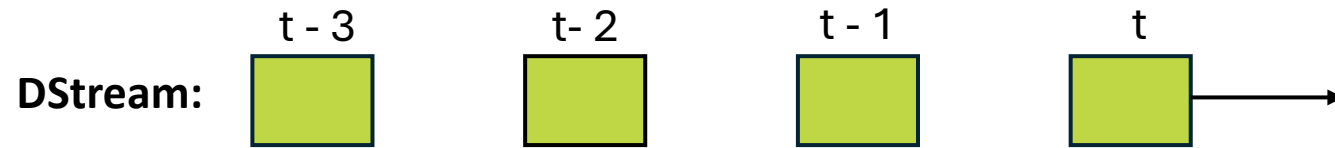


- Data Streaming
- Basic Concepts of Spark Streaming
- DStream Abstraction
 - Discretized Stream
 - DStream Interface
 - Example: Mapped DStream
 - Example: Windowed DStream
 - Example: Network Input Dstream
 - DStream Graph



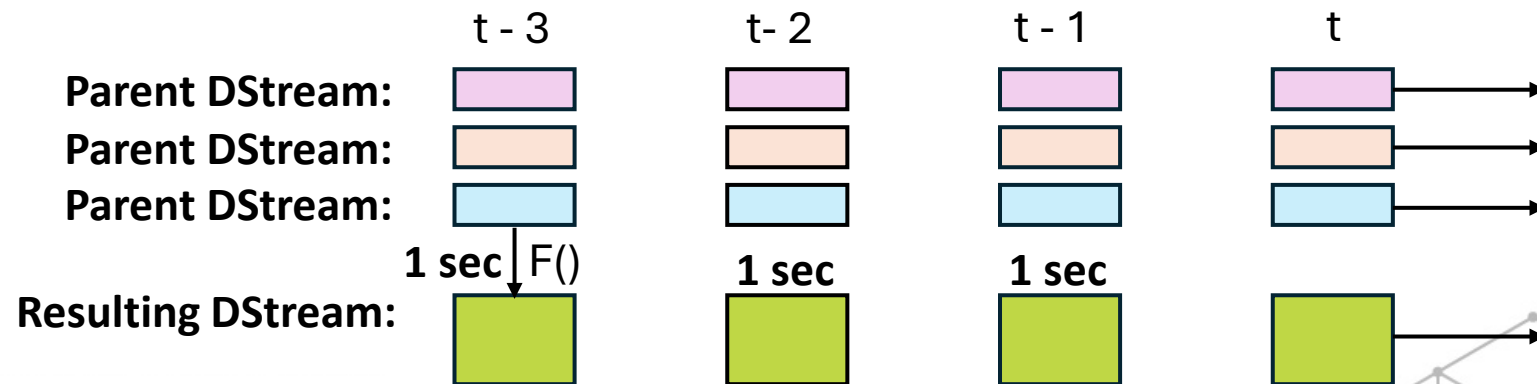
Discretized Stream

- A sequence of RDDs representing a stream of Data



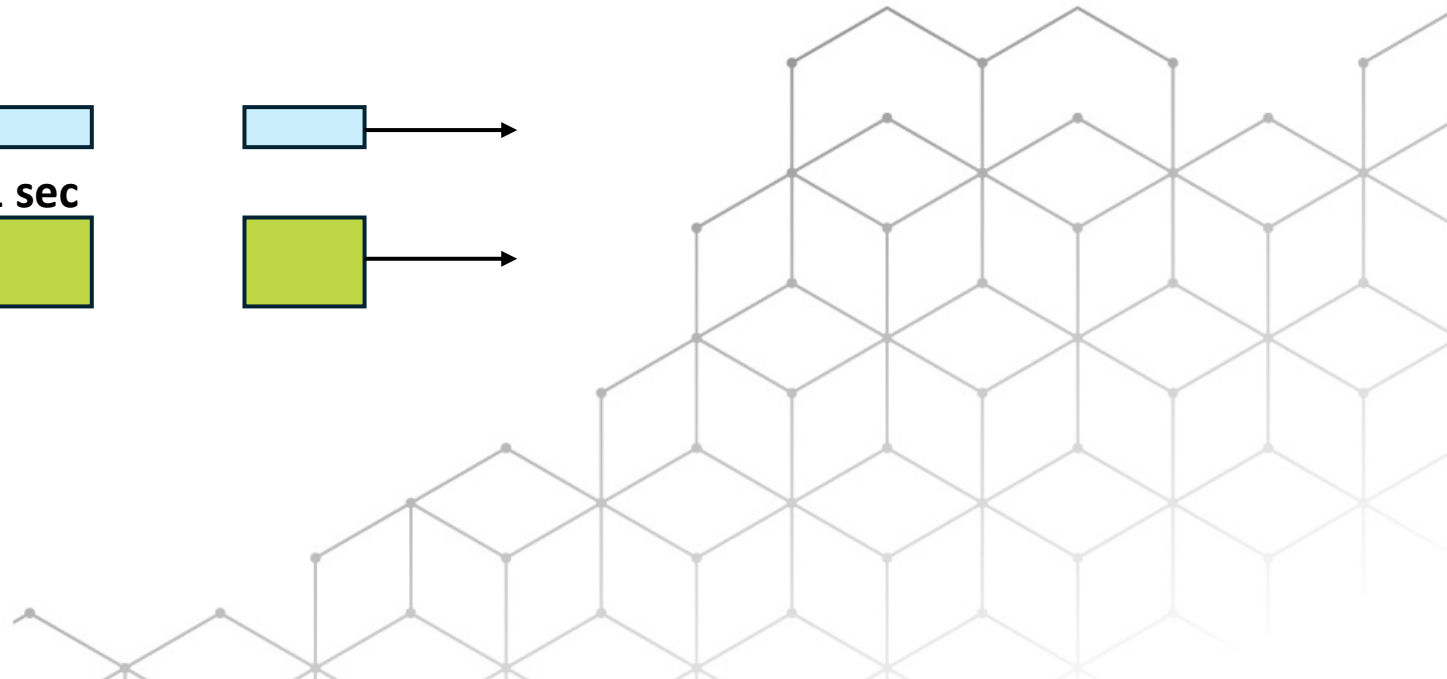
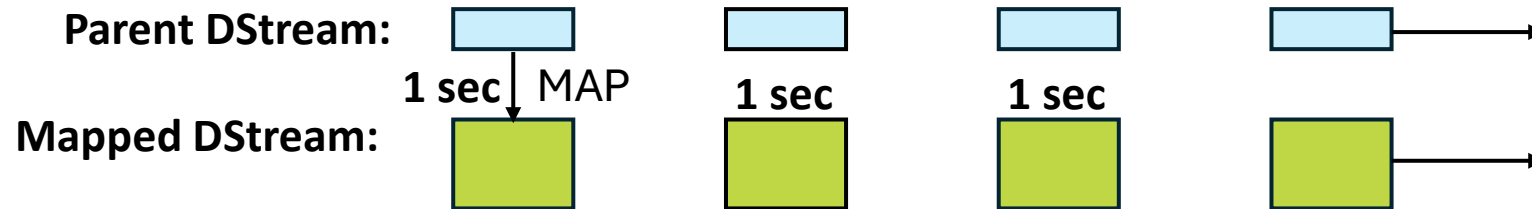
DStream Interface

- The DStream interface primarily defines how to generate an RDD in each batch interval:
 - List of dependent (Parent) DStreams
 - Slide Interval, the interval at which it will compute RDDs
 - Function to compute RDD at a time t



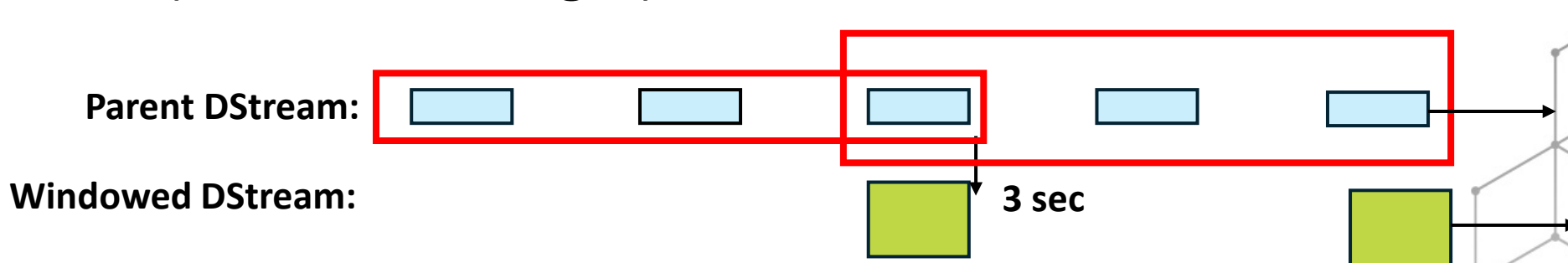
Mapped DStream

- Dependencies: Single Parent DStream
- Slide Interval: Same as parent Dstream (t)
- Create new RDD by applying map function on parent RDD of time t



Windowed Interface

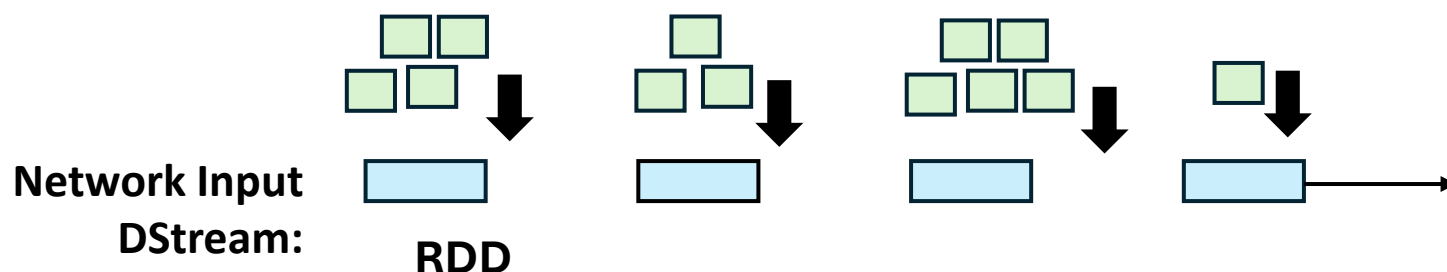
- Dependencies: Single Parent DStream
- Slide Interval: Window Sliding Interval (2 units of data)
- Apply union over all the RDDs of parent DStream between time t and $(t - \text{window length})$



The operation is applied over the last 3 time units of data, also slides by 2-time units.

Network Input DStream

- Base class of all input DStreams that receive data from the network
- Dependencies: None
- Slide Interval: Batch duration in streaming context
- Compute function for time t: Create a Block RDD with all the blocks of data received in the last batch interval



DStream Graph

- The DStream Graph represents the lineage or logical dependencies of operations in a Spark Streaming application.
- It describes how input data flows through various transformations and outputs.

```
t = ssc.twitterStream(...)
    .map(...)
t.foreach(...)
```



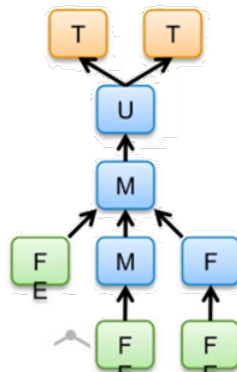
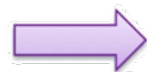
Dummy DStream signifying
an output operation



```
t1 = ssc.twitterStream(...)
t2 = ssc.twitterStream(...)

t = t1.union(t2).map(...)

t.saveAsHadoopFiles(...)
t.map(...).foreach(...)
t.filter(...).foreach(...)
```



Questions and Thank you

